

4	Top 6 Load..... حالات الاستخدام الستة الرئيسية لموزع الحمل
4	Balancer Use Cases).....
4	(Load Balancing Algorithms Categories)..... تصنيفات خوارزميات توزيع الحمل
4	في الشبكات Nginx ثالثاً: وظائف وأهمية خادم
5	رابعاً: الشرح التفصيلي للخوارزميات المتقدمة (مفهوماً وبرمجياً)
5	1. الخوارزمية المجدولة الموزونة (Weighted Round-Robin LB).....
5	2. التجزئة المتسقة (Consistent Hashing LB).....
5	3. الجلسات اللزجة / ألفة الجلسة (Sticky Sessions / Session Affinity LB).....
6	4. التوجيه القائم على زمن الاستجابة (Latency-Based Routing LB).....
6	5. التوزيع الواعي بالموارد (Resource-Aware LB).....
6	6. الخوارزميات التكيفية والقائمة على التغذية الراجعة (Adaptive / Feedback-Controlled).....
6	7. خوارزمية رتل الانضمام للحمول (Join-Idle-Queue - JIQ LB).....
6	8. توزيع الحمل في شبكات الخدمات (Load Balancing in Server Meshes).....
9	1. معالجة الرسائل والموثوقية (Messaging & Reliability).....
9	2. الأنظمة الموزعة (Distributed Systems).....
9	3. الانتقال للخدمات المصغرة (Monolith to Microservices).....
9	4. التوسع والتعامل مع الضغط (Scalability & Idempotency).....
9	5. الأمان والأداء (Security & Latency).....
10	6. تحسين بروتوكولات الاتصال (gRPC & REST).....
10	7. الإنتاجية ومعالجة البيانات (Batching & Backpressure).....
10	(Load Balancing Algorithms):..... أهم 6 خوارزميات لموازنة التحميل
10	(Static Algorithms)..... أولاً: الخوارزميات الثابتة
10	(Dynamic Algorithms)..... ثانياً: الخوارزميات الديناميكية
11	وكيفية عمله: Nginx أسباب شهرة
11	(Master-Worker Process Model)..... نموذج العمل
11	مميزاً: Nginx الميزات الرئيسية التي تجعل
11	(Senior):..... إلى مطور خبير (Junior) خارطة الطريق المكونة من 11 خطوة للتحويل من مطور مبتدئ
11	...وهي كالتالي: (Heartbeat) "في الأنظمة الموزعة باستخدام تقنية "نبضات القلب (nodes) آليات الكشف عن فشل العُقد
12	عند حدوث فشل في الأنظمة الموزعة، وهي تقنيات حيوية للتعامل مع (Retry Strategies) استراتيجيات إعادة المحاولة
13	الأخطاء المؤقتة وعدم استقرار الشبكة:.....
13	ما هو gRPC؟.....
13	(Pros & Cons)..... المميزات والعيوب
13	كيف يعمل؟ (الخصائص الرئيسية).....
14	(Use Cases)..... حالات الاستخدام
14	آليات اكتشاف فشل العُقد في الأنظمة الموزعة :.....
14	1. نبضات القلب المعتمدة على الدفع (Push-Based Heartbeat).....
14	2. نبضات القلب المعتمدة على السحب (Pull-Based Heartbeat).....
14	3. نبضات القلب مع فحص الصحة (Heartbeat with Health Check).....
14	4. نبضات القلب مع الطوابع الزمنية (Heartbeat with Timestamps).....
15	5. نبضات القلب مع التأكيد (Heartbeat with Acknowledgement).....
15	6. نبضات القلب مع النصاب (Heartbeat with Quorum).....
15	Reverse Proxy، API Gateway، و Load Balancer الفروقات بين الـ

1. مغير الهوية - (Reverse Proxy) الوكيل العكسي	15
2. ساعي البريد - (API Gateway) بوابة واجهة برمجة التطبيقات	15
3. شرطي المرور - (Load Balancer) موازن الحمل	15
..... باختصار: متى تستخدم أيًا منها؟	15
6 أنماط لإدارة البيانات (Data Management Patterns) أهم	16
1. الكاش الجانبي (Cache Aside)	16
2. العرض المَحَقَّق (Materialized View)	16
3. فصل أوامر القراءة والكتابة (CQRS)	16
4. مصادر الأحداث (Event Sourcing)	16
5. جدول الفهرسة (Index Table)	16
6. التجزئة (Sharding)	16
4 خوارزميات لتقسيم البيانات (Data Sharding Algorithms) أهم	17
1. التقسيم القائم على النطاق (Range-Based Sharding)	17
2. التقسيم القائم على التجزئة (Hash-Based Sharding)	17
3. التجزئة المتسقة (Consistent Hashing)	17
4. التجزئة عبر الصناديق الافتراضية (Virtual Bucket Hashing)	17
آلية عمل تقنية gRPC	18
..... RPC (Remote Procedure Call) مفهوم الـ	18
..... التقليدي؟ JSON أسرع بـ 5 مرات من نظام gRPC لماذا يعتبر	18
..... (gRPC Overall Flow) خطوات تدفق البيانات بالتفصيل	18
9 ممارسات لبناء أنظمة الخدمات المصغرة (Microservices) أهم	18
..... وكيفية حدوث ذلك في مراحل مختلفة Kafka احتمالية فقدان الرسائل في نظام	19
1. مرحلة المُنتِج (Producer)	19
2. مرحلة الوسيط (Broker)	19
3. مرحلة المستهلك (Consumer)	19
Kafka أهم 5 حالات استخدام لنظام	20
في تصميم الأنظمة (The Fantastic Four) "الأربعة الرائعون"	20
1. القابلية للتوسع (Scalability)	20
2. التوافرية (Availability)	20
3. الموثوقية (Reliability)	20
4. الأداء (Performance)	21
يوضح هذا الملف التنظيم الهيكلي الذي وضعه معيار (Linux File Systems) أهم المجلدات في نظام ملفات لينكس	
..... في عام 1994 لضمان اتساق النظام: (FHS) التسلسل الهرمي لنظام الملفات	21
..... المجلدات الأساسية ووظائفها:	21
في الأنظمة البرمجية (Latency) أهم 6 استراتيجيات لتقليل وقت الاستجابة	22
ضرورية لتصميم أنظمة عالية (Distributed Systems) هذه المفاهيم هي الركائز الأساسية في هندسة الأنظمة الموزعة	22
(High Availability) التوافر	22
1. الخطأ البيزنطي (Byzantine Fault)	22
2. عنق الزجاجة (Bottleneck)	23
3. الرسالة السامة (Poison Pill)	23
4. انقسام الدماغ (Split-Brain)	23
5. نمط العنقاء (Phoenix Pattern / Immutable Image)	23
حل النموذج المغلق 1	23
..... بيتزا Scaling السؤال الأول: تحديات الـ	23

.....	24
.....	24
.....	24
حل النموذج المغلق 2	25
.....	25
.....	25
.....	26
.....	26
.....	26
.....	26
.....	26
.....	26
خوارزمية Raft (Raft Consensus Algorithm)	27
.....	27
.....	27
.....	27
.....	27
.....	27
الاكواد	28
.....	28
.....	28
.....	29
.....	30
.....	30
.....	31
.....	32
.....	32
.....	33
.....	33
.....	35
.....	35
.....	36
.....	37
.....	37
.....	38
.....	38
.....	39
.....	39
.....	39
.....	39
.....	40

حالات الاستخدام الستة الرئيسية لموزع الحمل (Top 6 Load Balancer Use Cases)

يعمل موزع الحمل كعنصر أساسي في البنية التحتية للأنظمة الموزعة لتحقيق الوظائف التالية:

- توزيع حركة المرور (Traffic Distribution): توزيع طلبات المستخدمين بنسب متوازنة ومحددة (مثل: 30%، 40%، 30%) بين الخوادم لتقليل الضغط ومنع انهيار أي خادم.
- الإتاحة العالية (High Availability): توفير آلية نقل الخدمة (Failover) لخادم بديل وصحي فوراً في حال تعطل الخادم الأساسي لضمان استمرار عمل النظام دون انقطاع.
- إنهاء تشفير (SSL Termination): فك تشفير اتصالات HTTPS القادمة من المستخدمين عند موزع الحمل نفسه، ثم تمريرها كـ HTTP عادي إلى الخوادم الخلفية، مما يخفف عبء العمليات الحسابية للتشفير عن خوادم التطبيقات.
- استمرارية الجلسة (Session Persistence / Sticky Sessions): توجيه جميع الطلبات التابعة لمستخدم معين إلى الخادم الخلفي نفسه طوال فترة جلسته للحفاظ على بيانات الجلسة (مثل سلة التسوق أو حالة تسجيل الدخول).
- القابلية للتوسع (Scalability): تسهيل إضافة خوادم جديدة للشبكة بسلاسة تامة وتوجيه حركة المرور إليها للتعامل مع زيادة الطلب المتنامي.
- مراقبة الصحة (Health Monitoring): إرسال إشارات فحص نبضات القلب (Heartbeats) باستمرار للتأكد من سلامة الخوادم وعزل أي خادم متعطل تلقائياً.

ثانياً: تصنيفات خوارزميات توزيع الحمل (Load Balancing Algorithms) (Categories)

تنقسم الخوارزميات المستخدمة لتوزيع الأحمال إلى فئتين رئيسيتين:

1. الخوارزميات الثابتة (Static Algorithms): تعتمد على قواعد توجيه محددة مسبقاً دون النظر لحالة الخادم الحالية، وتضم:
 - خوارزمية Round-Robin (الدائرية المجدولة).
 - خوارزمية Sticky Round-Robin (الدائرية للزجة).
 - خوارزمية Weighted Round-Robin (الدائرية الموزونة).
 - خوارزمية Hash (التجزئة المبنية على خصائص معينة).
2. الخوارزميات الديناميكية (Dynamic Algorithms): تأخذ في الحسبان حالة الخوادم وأدائها في الوقت الحقيقي، وتضم:
 - خوارزمية Least Connections (الأقل اتصالاتاً).
 - خوارزمية Least Response Time (الأقل في زمن الاستجابة).

ثالثاً: وظائف وأهمية خادم Nginx في الشبكات

يتمتع Nginx بشعبية هائلة لأنه يتجاوز كونه خادم ويب عادي ليؤدي أدواراً متقدمة تشمل:

- الوكيل العكسي (Reverse Proxy): يعمل كوسيط يتلقى الطلبات من العميل ويمررها للخوادم الخلفية (App Servers)، ويتعامل مع اتصالات SSL والتخزين المؤقت المحلي (Local Cache).
- مخزن مؤقت للأصول الثابتة (Static Asset Cache): يقوم بتقديم الملفات الثابتة مباشرة (مثل صور jpg و ملفات التنسيق css) من كاش الـ Nginx دون إشغال خوادم التطبيقات، ويمرر فقط الطلبات الديناميكية (مثل ملفات البرمجة والـ APIs) إلى خوادم المحرك.

- **موزع حمل ذكي (Load Balancer):** يعتمد على محرك قواعد (Rules Engine) وخوارزميات متطورة لتوجيه حركة المرور بكفاءة إلى مجموعة الخوادم.
- **بوابة لمنع تكديس الكاش (Caching & Gateway):** يحتوي على خاصية "قفل المخزن المؤقت" (Cache Lock) التي تمنع حدوث مشكلة "تكديس الكاش" (Cache Stampede) عند انتهاء صلاحية الكاش، وبذلك يحمي قاعدة البيانات المركزية (SQL) من الانهيار بفعل الطلبات المتزامنة الضخمة.
- **بوابة متعددة السحابات وجدار حماية (Multi-Cloud Gateway & Firewall):** يوفر فحص النطاق الجغرافي للـ IP للعملاء (GeoIP Lookup)، وفلتر نشطة لحركة المرور عبر جدار حماية التطبيقات (WAF Filter)، والتخفيف من هجمات حجب الخدمة (DDoS Mitigation)، بالإضافة إلى دعم التوسع الأفقي التكيفي الآلي (Auto-adaptive Horizontal Scaling) عبر السحابات المختلفة.

رابعاً: الشرح التفصيلي للخوارزميات المتقدمة (مفهوماً وبرمجياً)

1. الخوارزمية المجدولة الموزونة (Weighted Round-Robin LB)

- **المفهوم:** يتم توزيع الطلبات بشكل متناسب بناءً على أوزان (Weights) محددة مسبقاً لكل خادم. تعكس هذه الأوزان قدرة الخادم الاستيعابية، أو أولويته، أو مواصفات العتاد (Hardware Tier) الخاص به.
- **مثال توضيحي:** إذا كانت الأوزان كالآتي: خادم A (وزن: 4)، خادم B (وزن: 2)، خادم C (وزن: 1)، خادم D (وزن: 3)، فإن مجموع الأوزان هو 10 طلبات في الدورة الواحدة. بناءً على ذلك، يتلقى الخادم A نسبة 40% من حركة المرور، وتتوزع البقية بنسب (20%، 10%، 30%) على التوالي بالتناسب التام مع أوزانها.
- **المنطق البرمجي (Code Logic):** تعتمد الخوارزمية برمجياً على مؤشر ذري يتزايد تلقائياً مع كل طلب (AtomicInteger index)، ويتم حساب باقي قسمته على مجموع أوزان الخوادم الكلي (totalWeight). تمر حلقة التكرار على الخوادم وتجمع أوزانها تراكمياً؛ وعندما تكون قيمة المؤشر الحالية أقل من المجموع التراكمي للأوزان في تلك اللحظة، يتم اختيار هذا الخادم فوراً لإرسال الطلب إليه.

2. التجزئة المتسقة (Consistent Hashing LB)

- **المفهوم:** تقوم هذه الخوارزمية برسم خرائط للطلبات والخوادم معاً على حلقة تجزئة افتراضية واحدة (Virtual Hash Ring) تتراوح زواياها من 0 إلى 360 درجة. يتم توجيه طلب العميل باتجاه عقارب الساعة حتى يصل إلى أول خادم يواجهه على الحلقة.
- **الميزة الأساسية:** عند انضمام خادم جديد أو مغادرة خادم موجود، يتم إعادة توزيع جزء ضئيل جداً فقط من المفاتيح (يتناسب مع النسبة $\$N/1\$$ حيث $\$N\$$ هو عدد الخوادم)، بينما تظل خرائط وتوجيهات الخوادم الأخرى مستقرة دون أي تغيير نظامي شامل.
- **بنية الإصلاح الذاتي والتوسع الافتراضي:** لمنع حدوث بقع ساخنة وعشوائية في توزيع الحمل (Overload)، تستخدم الخوارزمية التكرارية تقنية "العقد الافتراضية" (Virtual Nodes) المرمزة مثل `A_vNode1` و `C_vNode1`؛ حيث يمثل الخادم الحقيقي عدة نقاط على الدائرة لضمان توزيع عادل ومثالي للأحمال، ولتسهيل عزل الخوادم الفاشلة تلقائياً.
- **المنطق البرمجي:** يتم تمثيل الحلقة برمجياً باستخدام الخريطة المرتبة `TreeMap` (المعروفة بـ `SortedMap`). عند وصول طلب، يُحسب الهاش الخاص بمفتاح الطلب (Request Key) ثم يُبحث عن أول خادم يملك قيمة هاش أكبر أو تساوي هاش الطلب باستخدام `tailMap(key)`؛ وإذا كانت النتيجة فارغة، يتم إرجاع أول خادم في الدائرة (بداية الحلقة).

3. الجلسات اللزجة / ألفة الجلسة (Sticky Sessions / Session Affinity LB)

- **المفهوم:** تضمن هذه الخوارزمية توجيه كافة الطلبات اللاحقة التابعة لنفس الجلسة إلى الخادم الخلفي ذاته الذي استقبل الطلب الأول، طوال فترة صلاحية الجلسة (Session's Lifetime).
- **مثال تشبيهي:** يشبه النظام موظف البنك (Teller) الذي قام بفتح ملف العميل والتحقق من هويته ومستنداته بالفعل؛ فعند عودة العميل لإجراء معاملة أخرى، يتم توجيهه مباشرة لنفس الموظف لتوفير الوقت وتجنب تكرار خطوات الأمان والتحقق.
- **الآلية وفشل الخادم:** يتم تتبع ومعرفة الارتباط عن طريق جدول جلسات (Session Database) يعتمد على ملفات تعريف الارتباط الرمزية (Cookie/Token). وفي حالة تعطل الخادم المخصص، تنتهي الألفة (Affinity Broken) ويتم إعادة توجيه العميل (Re-hash) وتعيين خادم جديد له.

- **المنطق البرمجي:** تُستخدم بنية بيانات خريطة متزامنة من نوع `ConcurrentHashMap` لتخزين معرف الجلسة ك مفتاح (Key) واسم الخادم المعين ك قيمة (Value). وتُستدعى الدالة `computeIfAbsent` لضمان تعيين خادم جديد عبر الدائرية المجدولة (Round-Robin) فقط إذا لم تكن الجلسة مسجلة مسبقاً في الذاكرة.

4. التوجيه القائم على زمن الاستجابة (Latency-Based Routing LB)

- **المفهوم:** يوجه موزع الحمل الطلبات الواردة ديناميكياً إلى الخادم الذي سجل أقل زمن استجابة (Lowest Response Time / RTT) في الفترة الأخيرة.
- **حلقة التغذية الراجعة الديناميكية:** يقوم النظام بإرسال مجسات فحص الكمون (Latency Probes) بانتظام لمراقبة أوقات الاستجابة وحالة الاتصال للتعرف على أي طفرات بطء أو أخطاء حرجية، مما يتيح له عزل وتصريف وتوجيه حركة المرور بعيداً عن الخوادم المجهدة والمسارات عالية التأخير (High Latency Paths).
- **المعادلة الرياضية للتحديث:** يتم حساب التحديث لزمن الاستجابة المتوقع عبر تطبيق المتوسط المتحرك الأسّي (Exponential Moving Average) بالاستعانة بمعامل تنعيم $\alpha = 0.2$ على سبيل المثال) وفق الصيغة التالية:
$$\text{latency} = \alpha \times \text{measured_ms} + (1 - \alpha) \times \text{latency}$$
حيث تضمن هذه الصيغة إعطاء وزن أكبر للقياسات الحديثة مع الحفاظ على استقرار السلوك التاريخي للخادم.

5. التوزيع الواعي بالموارد (Resource-Aware LB)

- **المفهوم:** يوزع حركة المرور بالاعتماد على مراقبة واستهلاك الموارد الفعلي للخوادم في الوقت الحقيقي (مثل استهلاك المعالج CPU، الذاكرة RAM، عمليات الإدخال والإخراج I/O، والـ Threads).
- **الآلية:** يختار الموزع دائماً الخادم صاحب أقل عبء عمل حالي (Lowest Workload). كما يمكن للخوارزمية تصفية وفلترة الخوادم المؤهلة (Eligible Nodes) بناءً على متطلبات الطلب المعقدة؛ مثل التحقق مما إذا كان الطلب يتطلب حجماً معيناً من الذاكرة أو يحتاج إلى معالج رسومي (GPU)، ثم يختار الأقل استهلاكاً للمعالج من بين الخوادم المؤهلة فقط.

6. الخوارزميات التكيفية والقائمة على التغذية الراجعة (Adaptive / Feedback-Controlled)

- **المفهوم:** تعدل استراتيجيات التوجيه أو الأوزان المخصصة للخوادم تلقائياً وبشكل مستمر بناءً على التغذية الراجعة لأداء الخوادم (مثل معدلات النجاح، ومعدلات الأخطاء، وعمق رتل الطلبات).
- **آلية العمل:** يقوم النظام بفحص متواصل لنسب الأخطاء؛ فإذا تجاوزت الأخطاء حداً معيناً (مثل 5 أخطاء)، يتم معاقبة الخادم (Penalize) بخفض وزنه تلقائياً لحمايته من الانهيار، وفور استقراره وعودته للوضع الطبيعي يتم تعافيه (Recover) برفع وزنه تدريجياً ليعود لاستقبال الحجم الكامل للبيانات.

7. خوارزمية رتل الانضمام للخمول (Join-Idle-Queue - JIQ LB)

- **المفهوم:** تعتمد على مبدأ ذكي يقوم فيه الخادم نفسه بإرسال إشارة صريحة لموزع الحمل عندما يصبح خاملاً وفارغاً (Idle). يقوم الموزع بتسجيل هذا الخادم في رتل خاص، ولا يتم إرسال الطلبات إلا للخوادم التي قامت بتسجيل نفسها طواعية كـ "متاحة"، مما يقضي تماماً على تراكم رتل الطلبات بداخل الخوادم.
- **المنطق البرمجي:** تستخدم الخوارزمية رطلاً حاجباً ومتزامناً (`BlockingQueue`) مثل `LinkedBlockingQueue`. عندما يصبح الخادم خاملاً، يقوم باستدعاء دالة تقدم معرفه للرتل (`idleQueue.offer(serverId)`). وعند وصول طلب جديد، يقوم موزع الحمل بسحب أول خادم متاح من الرتل باستخدام دالة السحب (`idleQueue.take()`) لمعالجة الطلب فوراً.

8. توزيع الحمل في شبكات الخدمات (Load Balancing in Server Meshes)

- **المفهوم:** يتم فصل منطق التوجيه وتوزيع الحمل تماماً عن كود التطبيق الأساسي من خلال زرع بروكسي جانبي (Sidecar Proxies مثل Envoy) ليعترض حركة المرور ويتولى مهام توزيع الحمل، وإعادة المحاولة عند الفشل (Retries)، وإدارة المهل الزمنية (Timeouts)، وتفعيل قاطع الدائرة (Circuit Breaking).
- **التحكم وبنية الإصلاح الذاتي:** تدار هذه البروكسيات مركزياً عبر لوحة تحكم شبكة الخدمات (Service Mesh Control Plane مثل Istio). تقوم لوحة التحكم بجمع بيانات الصحة الفعالة باستمرار لعزل وتصريف الخوادم ذات

معدلات الخطأ العالية (Failed Node Draining)، مما يحقق بيئة إصلاح ذاتي كاملة تضمن استعادة القدرة التشغيلية عبر الأقاليم السحابية المختلفة دون تأثير النظام.

ملخص المفاهيم المعمارية للامتحان

المفهوم

الفرق الجوهرى

Monolith vs. Modular Monolith

المونوليث (Monolith) كود واحد مترابط. المودولار مونوليث يقسم النظام لوحدات منطقية مستقلة (Modules) داخل نفس قاعدة الكود، مما يسهل الانتقال لاحقاً للخدمات المصترة (Microservices).

Orchestration vs. Choreography

في الأوركسترا (Orchestration) يوجد "مُنسق" مركزي يتحكم بتنسيق العمل. في الكوريوترافيا (Choreography) تتواصل الخدمات مع بعضها عبر أحداث (Events) دون وجود قائد مركزي.

Kafka vs. RabbitMQ

كافكا (Kafka) نظام سجل أحداث (Event Streaming) عالي الإنتاجية وتخزيني. رايبيت إم كيو (RabbitMQ) وسيط رسائل تقليدي يركز على توجيه الرسائل بدقة (Message Routing).

Synchronous vs. Asynchronous

الاتصال المتزامن (REST) ينتظر رد الخدمة فوراً (حظر). غير المتزامن (Queue) يرسل الرسالة ويتابع العمل دون انتظار (غير محظور).

Strong vs. Eventual Consistency

القوة (Strong) تضمن أن كل نسخة من البيانات محدثة فوراً (ضحية بالسرعة). التوافق النهائي (Eventual) يسمح بتأخير بسيط في التحديث عبر النظام مقابل توفر وسرعة أعلى.

Outbox vs. CDC

نمط الـ Outbox يحفظ الأحداث في جدول بقاعدة البيانات كجزء من المعاملة (Transaction). الـ CDC (Change Data Capture) يراقب سجلات قاعدة البيانات ويرسل الأحداث تلقائياً عند تغييرها.

Gossip Protocol vs. Centralized

بروتوكول Gossip يوزع المعلومات بين العقد بطريقة عشوائية (مثل إشاعة). الاكتشاف المركزي (Centralized) يعتمد على خادم واحد يمتلك قائمة بكل العقد في النظام.

NATS vs. MQTT

NATS خفيف وسريع جداً للمراسلة بين الخدمات. MQTT صُمم خصيصاً لإنترنت الأشياء (IoT) والاتصالات ضعيفة الاستقرار.

DLX vs. Regular Queue

الـ DLX (Dead Letter Exchange) هو طابور خاص يستقبل الرسائل التي فشلت معالجتها في الطابور العادي (Regular Queue) لتحليلها أو إعادة المحاولة.

Redis vs. Memcached

ريديس (Redis) يدعم هياكل بيانات متنوعة ويدعم التخزين الدائم (Persistence). ميكايش (Memcached) بسيط جداً، سريع، ويستخدم للذاكرة المؤقتة فقط (Cache) دون هياكل معقدة.

سريع، ويسمح للذاكرة المؤقتة فقط (Cache) بكون هيكل معتدلة.	
التقسيم (Partitioning) هو تقسيم البيانات عمودياً أو أفقياً داخل نفس المكان. الـ Sharding هو توزيع هذه الأجزاء على خوادم مختلفة. الـ Replication هو نسخ البيانات نفسها على خوادم متعددة لزيادة التوافر.	Sharding vs. Partitioning vs. Replication
Token Bucket يحدد معدل الطلبات عبر "دلو" يمتلئ بـرموز (Tokens). Sliding Window يتتبع الطلبات ضمن نافذة زمنية متحركة لضمان دقة أكبر في تحديد معدل الطلب (Rate Limiting).	Token Bucket vs. Sliding Window

1. معالجة الرسائل والموثوقية (Messaging & Reliability)

- **Transactional Outbox Pattern**: لحل ضياع الرسائل، لا ترسل الرسالة مباشرة بعد قاعدة البيانات. بدلاً من ذلك، احفظ "الحدث" (Event) في جدول **Outbox** داخل نفس المعاملة (Transaction) مع تحديث البيانات. ثم يقوم **Message Relayer** بقراءة الجدول وإرسال الرسائل للـ **Broker** (مثل RabbitMQ).
- **DLX (Dead Letter Exchange)**: هو "طابور الأخطاء". إذا فشلت معالجة رسالة (بسبب بيانات تالفة)، تُنقل للـ **DLX**. آلية إعادة المحاولة: يتم إرجاع الرسالة من الـ **DLX** إلى الطابور الأصلي بعد وقت محدد (TTL) باستخدام **x-message-ttl** و **x-dead-letter-exchange** حتى حد معين، بعدها تُحفظ نهائياً في طابور الأخطاء اليدوي.

2. الأنظمة الموزعة (Distributed Systems)

- **Gossip Protocol**: يستخدم في الأنظمة الضخمة (500 عقدة) لأن كل عقدة تتحدث مع جيرانها فقط (مثل الإشاعة). الميزة: لا توجد نقطة فشل واحدة (Single Point of Failure) كما في **Centralized Registry** (مثل Zookeeper) الذي قد يصبح عنق زجاجة.
- **Redis Distributed Lock**: لمنع تضارب التعديل. يُستخدم **SET resource_name NX PX 30000 my_random_value**. **المخاطر**: في حال توقف العميل بسبب GC (Garbage Collection) قد ينتهي القفل ويأخذه عميل آخر. الحل: استخدام **Redlock** أو **Fencing Tokens** (رقم تسلسلي للتأكد من أحدث عميل).

3. الانتقال للخدمات المصغرة (Monolith to Microservices)

- **مشاكل JavaRMI**: مشكلة في الـ Cloud بسبب الـ Firewall (تتطلب فتح منافذ كثيرة)، غير متوافقة مع الـ Load Balancers، وتتطلب وجود JVM على الطرفين. الحل: التحول لـ REST أو gRPC مع JSON/Protobuf.
- **Cache Invalidation**: عند تحديث فيديو، نستخدم **Cache Tagging** أو **Versioned URLs** (تغيير رابط الفيديو فور تحديثه) ليجبر الـ CDN على جلب النسخة الجديدة فوراً.

4. التوسع والتعامل مع الضغط (Scalability & Idempotency)

- **Overselling**: لتجنب البيع المزدوج، استخدم **Redis Lua Scripts** (تضمن ذرية العملية - Atomicity - في خطوة واحدة) بدلاً من **SELECT...FOR UPDATE** التي قد تبطئ النظام في حال الضغط المليوني.
- **Idempotency**: لتجنب تكرار العمليات، يولد التطبيق (UUID) **(Idempotency-Key)** عند بداية الطلب. يحفظ الـ API Gateway هذا المفتاح في Redis. إذا وصل نفس الطلب (مع نفس المفتاح)، يُرفض أو يُعاد نفس الرد السابق.

5. الأمان والأداء (Security & Latency)

- **Nginx Tuning**
 - **limit_req_zone**: يحدد عدد الطلبات في الثانية لمنع الـ Brute force.
 - **client_body_timeout**: يقطع الاتصالات التي ترسل البيانات ببطء شديد (حماية ضد Slowloris).

- **Tail Latency**: لحل بطء خدمة واحدة، نستخدم **Circuit Breaker** (لقطع الاتصال عن الخدمة البطيئة فوراً) و **Hedged Requests** (إرسال الطلب لنسختين من الخدمة وأخذ الرد الأسرع).

6. تحسين بروتوكولات الاتصال (gRPC & REST)

- **gRPC vs REST**: الـ JSON يحتوي **Overhead** كبير (gRPC). Headers يستخدم **Protocol Buffers** (ثنائي، صغير الحجم) و **HTTP/2 Multiplexing** (إرسال عدة طلبات عبر اتصال واحد)، مما يقلل الـ Latency بشكل كبير.
- **Real-time**: نستخدم **Bidirectional Streaming** لتحديث المخزون لحظياً. لإدارة الانقطاع، نستخدم **Exponential Backoff** مع **Reconnection** مع الاحتفاظ بـ **Last-Received-ID** لإكمال البيانات المفقودة.

7. الإنتاجية ومعالجة البيانات (Batching & Backpressure)

- **Backpressure**: عندما تكون سرعة الكتابة في قاعدة البيانات أقل من سرعة الاستقبال، نستخدم طوابير (Queues) مع **Thresholds**. إذا امتلأ الطابور، نطبق **Dropping Strategy** (إسقاط البيانات غير الحساسة).
- **Batching**: نستخدم "جامع" (Buffer) ينتظر **Max_Records** (1000) أو **Max_Time** (50ms). للكتابة المتوازية مع حفظ الترتيب، نقوم بتقسيم البيانات بناءً على **Partition Key** (مثل معرف الجهاز) لضمان أن بيانات الجهاز الواحد تذهب دائماً لنفس الـ Batch.

أهم 6 خوارزميات لموازنة التحميل (Load Balancing Algorithms):

تنقسم هذه الخوارزميات إلى نوعين رئيسيين:

أولاً: الخوارزميات الثابتة (Static Algorithms)

هذه الخوارزميات لا تأخذ حالة النظام الحالية بعين الاعتبار عند توزيع الطلبات:

- **Round Robin (التوزيع الدوري)**: يتم إرسال طلبات العميل إلى مثيلات الخدمة بترتيب تسلسلي دوري. تتطلب هذه الطريقة أن تكون الخدمات عديمة الحالة (stateless).
- **Sticky Round Robin (التوزيع الدوري المترابط)**: تحسين للنوع السابق، حيث يضمن أنه إذا قام مستخدم (مثل "أليس") بإرسال طلبه الأول إلى خدمة معينة، فإن طلباته اللاحقة ستوجه إلى نفس الخدمة.
- **Weighted Round Robin (التوزيع الدوري الموزون)**: يسمح للمدير بتحديد "وزن" لكل خدمة. الخدمات ذات الوزن الأعلى تستقبل عدداً أكبر من الطلبات مقارنة بالخدمات الأخرى.
- **Hash (التجزئة)**: يعتمد على تطبيق دالة تجزئة (Hash function) على عنوان IP الخاص بالطلب أو الرابط (URL) الخاص به؛ بناءً على نتيجة التجزئة، يتم توجيه الطلب إلى المثل المناسب.

ثانياً: الخوارزميات الديناميكية (Dynamic Algorithms)

هذه الخوارزميات تقوم بتحليل حالة النظام الحالية لاتخاذ قرار التوزيع:

- **Least Connections (أقل الاتصالات)**: يتم إرسال الطلب الجديد إلى مثيل الخدمة الذي لديه أقل عدد من الاتصالات المتزامنة حالياً.

- **Least Response Time (أقل زمن استجابة):** يتم إرسال الطلب الجديد إلى مثيل الخدمة الذي يقدم أسرع زمن استجابة في تلك اللحظة.

أسباب شهرة Nginx وكيفية عمله:

يُعد Nginx خادم ويب (Web Server) ووكيل عكسي (Reverse Proxy) عالي الأداء، وتعود شهرته إلى بنيته الهندسية الفريدة التي تمنحه استقراراً وقابلية للتوسع واستغلالاً كفوئاً للموارد.

نموذج العمل: (Master-Worker Process Model)

يعتمد Nginx على نموذج "العمليات الرئيسية والعمال":

- **العملية الرئيسية (Master Process):** مسؤولة عن قراءة الإعدادات وإدارة عمليات العمال.
- **عمليات العمال (Worker Processes):** تتعامل مع الاتصالات الواردة باستخدام نموذج إدخال/إخراج غير محظور ومدفوع بالأحداث (Event-driven non-blocking I/O)، مما يتيح له التعامل مع عدد كبير من الاتصالات المتزامنة.

الميزات الرئيسية التي تجعل Nginx مميزاً:

1. **خادم ويب عالي الأداء (High-Performance Web Server):** قادر على التعامل مع عدد كبير من الاتصالات المتزامنة وتقديم المحتوى الثابت تحت أحمال حركة مرور عالية.
2. **الوكيل العكسي وموازنة الحمل (Reverse Proxy and Load Balancing):** يعمل كوسيط أمام خوادم الويب، حيث يقوم بتوزيع حركة المرور عبر خوادم خلفية متعددة (backend servers).
3. **التخزين المؤقت للمحتوى (Content Cache):** يُستخدم لتقديم الصفحات بسرعة فائقة من خلال استرجاع المحتوى من ذاكرة التخزين المؤقت.
4. **إنهاء الـ SSL (SSL Termination):** يوفر دعماً مدمجاً لـ SSL/TLS، مما يخفف عبء عملية فك التشفير عن الخوادم الخلفية.

هذه البنية المتكاملة تجعل من Nginx خياراً مفضلاً للعديد من تطبيقات الويب الحديثة التي تتطلب كفاءة وسرعة عاليتين.

خارطة الطريق المكونة من 11 خطوة للتحويل من مطور مبتدئ (Junior) إلى مطور خبير (Senior):

- **1 - أدوات التعاون (Collaboration Tools):** تطوير البرمجيات نشاط اجتماعي، لذا يجب تعلم أدوات مثل Jira، MS Teams، Slack، و Confluence.
- **2 - لغات البرمجة (Programming Languages):** قم باختيار وإتقان لغة أو لغتين من لغات البرمجة مثل JavaScript، Python، Java، أو Go.
- **3 - تطوير واجهات برمجة التطبيقات (API Development):** تعلم كيفية التعامل مع مناهج تطوير الـ API مثل REST، GraphQL، و gRPC.

- 4 - خوادم الويب والاستضافة (Web Servers and Hosting): تعرّف على خوادم الويب والمنصات السحابية مثل AWS، Azure، GCP، وKubernetes.
- 5 - المصادقة والاختبار (Authentication and Testing): تعلم تقنيات التحقق من الهوية والاختبارات البرمجية مثل TDD وUnit Test.
- 6 - قواعد البيانات (Databases): تعلم العمل مع قواعد البيانات العلائقية (Relational) وغير العلائقية (Non-relational) مثل PostgreSQL وMongoDB.
- 7 - التكامل والتسليم المستمر (CI/CD): تعلم أدوات وأتمتة عمليات بناء وتدفق البرمجيات لضمان كفاءة التسليم.
- 8 - هياكل البيانات والخوارزميات (Data Structures and Algorithms): ركّز على المفاهيم الأساسية مثل الـ Big O، Recursion، Sorting، الأشجار (Trees)، والرسوم البيانية (Graphs).
- 9 - تصميم النظم (System Design): تعلم كيفية بناء أنظمة قابلة للتوسع باستخدام تقنيات مثل الميكروسيرفيس (Microservices)، موازنة الحمل (Load Balancing)، والتخزين المؤقت (Caching).
- 10 - أنماط التصميم (Design Patterns): تعلم أنماط التصميم البرمجية مثل Factory، Proxy، Observer، وغيرها لتحسين جودة الكود.
- 11 - أدوات الذكاء الاصطناعي (AI Tools): استغل أدوات الذكاء الاصطناعي لزيادة إنتاجيتك، مثل GitHub Copilot، ChatGPT، وPrompt Engineering.

آليات الكشف عن فشل العُقد (nodes) في الأنظمة الموزعة باستخدام تقنية "نبضات القلب" (Heartbeat)، وهي كالتالي:

- **نبضات القلب القائمة على الدفع (Push-Based Heartbeat):**
 - تقوم العقدة بإرسال إشارة دورية إلى المراقب.
 - المميزات: بسيطة التنفيذ.
 - العيوب: قد تؤدي ازدحامات الشبكة أو التأخيرات إلى إنذارات كاذبة (false positives).
- **نبضات القلب القائمة على السحب (Pull-Based Heartbeat):**
 - يقوم المراقب بطلب (سحب) معلومات الحالة من العقدة دورياً.
 - المميزات: تقلل من حركة مرور الشبكة.
 - العيوب: قد تزيد من زمن الاستجابة (latency) في اكتشاف الفشل.
- **نبضات القلب مع فحص الصحة (Heartbeat with Health Check):**
 - توفر تفاصيل إضافية حول أداء العقدة (مثل مقاييس المعالج والذاكرة).
 - العيوب: تزيد من التعقيد والحمل على الشبكة.
- **نبضات القلب مع الطوابع الزمنية (Heartbeat with Timestamps):**
 - تساعد في اكتشاف زمن انتقال الشبكة.
 - العيوب: تتطلب مزامنة دقيقة للساعات بين الأجهزة.
- **نبضات القلب مع التأكيد (Heartbeat with Acknowledgement):**
 - يقوم المراقب بإرسال تأكيد (ACK) عند استلام النبضة للتأكد من حالة المرسل.
 - العيوب: تؤدي إلى مضاعفة عدد الرسائل المتبادلة.
- **نبضات القلب مع النصاب القانوني (Heartbeat with Quorum):**
 - تستخدم للحفاظ على سلامة النظام واتساقه.
 - العيوب: قد تكون صعبة في إدارة العقد التابعة.

استراتيجيات إعادة المحاولة (Retry Strategies) عند حدوث فشل في الأنظمة الموزعة، وهي تقنيات حيوية للتعامل مع الأخطاء المؤقتة وعدم استقرار الشبكة:

- **إعادة المحاولة الخطية (Linear Backoff):**
 - آلية العمل: يتم الانتظار لفترة زمنية ثابتة بين كل محاولة وأخرى.
 - المميزات: سهولة التنفيذ والفهم.
 - العيوب: ليست مثالية في حالات الضغط العالي أو التزامن الكبير، حيث قد تؤدي إلى ما يسمى بـ "عواصف إعادة المحاولة" (retry storms) وتنافس على الموارد.
- **إعادة المحاولة الخطية مع التذبذب (Linear Jitter Backoff):**
 - آلية العمل: تطوير للاستراتيجية الخطية عبر إضافة "عشوائية" (Randomness) لفترات الانتظار.
 - الهدف: تقليل مشاكل التزامن التي تحدث عند محاولة عدة عُقد إعادة الاتصال في نفس اللحظة بالضبط.
- **إعادة المحاولة الأسية (Exponential Backoff):**
 - آلية العمل: زيادة فترة الانتظار بين المحاولات بشكل أسي (مثلاً: ثانية، ثانيتان، 4 ثوانٍ، وهكذا).
 - المميزات: فعالة جداً في منح النظام وقتاً كافياً للتعافي.
 - العيوب: في حالات الأخطاء البسيطة التي يمكن حلها بسرعة، قد تؤدي هذه الاستراتيجية إلى تأخير غير ضروري في الحل.
- **إعادة المحاولة الأسية مع التذبذب (Exponential Jitter Backoff):**
 - آلية العمل: تدمج بين الزيادة الأسية في وقت الانتظار وبين إضافة العشوائية (Jitter).
 - الهدف: هي الاستراتيجية الأكثر قوة، حيث تمنع مشاكل التزامن بشكل فعال مع منح النظام فرصة كافية للتعافي دون التسبب في حمل زائد.
 - العيوب: قد تتسبب أحياناً في تأخيرات أطول مما هو ضروري بسبب العشوائية المضافة.

ما هو gRPC؟

هو إطار عمل مفتوح المصدر طوّره جوجل، مصمم لتسهيل التواصل الفعال والقوي بين الخدمات في بنية "الخدمات المصغرة" (Microservices)، ويُعتبر خياراً شائعاً لبناء الأنظمة الموزعة وواجهات برمجة التطبيقات (APIs).

المميزات والعيوب (Pros & Cons)

- **المميزات (Pros):**
 - أداء عالٍ جداً.
 - غير مقيد بلغة برمجة معينة (Language agnostic).
 - يستخدم واجهات محددة النوع بدقة (Strongly typed interfaces).
 - يدعم البث (Streaming).
 - فعال جداً في تسلسل البيانات (Efficient data serialization).
- **العيوب (Cons):**
 - تعقيد أكبر ومنحنى تعلم أصعب.
 - مجتمع دعم أصغر مقارنة بـ REST.
 - قيود محتملة مع بعض جدران الحماية (Firewalls) والوكلاء (Proxies).
 - محدودية في قراءة البشر للبيانات (مقارنة بـ JSON/XML).

كيف يعمل؟ (الخصائص الرئيسية)

1. تنسيق البيانات (Protocol Buffers): يستخدم gRPC ملفات proto لتحديد العقود (Contracts) بين الخدمات، مما يجعل الرسائل أصغر وأسرع بكثير من JSON أو XML.
2. النقل عبر HTTP/2: يعتمد بشكل أساسي على بروتوكول HTTP/2، مما يوفر تحسينات كبيرة في السرعة وإدارة الاتصالات مقارنة بـ HTTP/1.x.
3. دورة حياة الطلب: يمر الطلب بعمليات تشفير (Encoding) وفك تشفير (Decoding) عبر مكتبات gRPC قبل إرساله عبر الشبكة، مما يضمن كفاءة عالية في نقل البيانات بين العميل والخادم.

حالات الاستخدام (Use Cases)

- بناء أنظمة الخدمات المصغرة (Microservices).
- تطبيقات التحديث اللحظي (Live Update).
- أنظمة إنترنت الأشياء (IoT).
- البيئات التي تستخدم لغات برمجة متعددة (Polyglot Env).

آليات اكتشاف فشل العقد في الأنظمة الموزعة :

تعتبر مراقبة حالة العقد (Nodes) أمراً حيوياً لاستقرار الأنظمة الموزعة. تعتمد هذه العملية على إرسال إشارات دورية تُعرف بـ "نبضات القلب" (Heartbeats). فيما يلي ملخص شامل لأهم 6 آليات مستخدمة:

1. نبضات القلب المعتمدة على الدفع (Push-Based Heartbeat)

- كيف تعمل: تقوم العقدة بإرسال إشارة دورية تلقائياً للمراقب.
- المميزات: سهولة التنفيذ والبناء.
- العيوب: حساسة للازدحام في الشبكة؛ حيث قد يؤدي التأخير في وصول الإشارة إلى "إيجابيات كاذبة" (ظن النظام أن العقدة تعطلت بينما هي تعمل).

2. نبضات القلب المعتمدة على السحب (Pull-Based Heartbeat)

- كيف تعمل: يقوم المراقب "بسحب" (طلب) الحالة من العقد بشكل دوري.
- المميزات: تقلل من حركة المرور غير الضرورية في الشبكة.
- العيوب: قد تؤدي إلى تأخير (Latency) في اكتشاف الفشل الفعلي للعقدة.

3. نبضات القلب مع فحص الصحة (Heartbeat with Health Check)

- كيف تعمل: لا تكتفي العقدة بإرسال إشارة الحياة، بل ترفق معها تفاصيل تقنية (مثل استهلاك المعالج CPU، استخدام الذاكرة، أو مقاييس خاصة بالتطبيق).
- المميزات: توفر رؤية أعمق لصحة العقدة، مما يساعد في اتخاذ قرارات ذكية (مثل إعادة توجيه المهام قبل وقوع الفشل).
- العيوب: تزيد من تعقيد النظام وتستهلك موارد شبكة أكبر.

4. نبضات القلب مع الطوابع الزمنية (Heartbeat with Timestamps)

- كيف تعمل: يتم تضمين وقت إرسال الإشارة في النبضة نفسها.
- المميزات: تساعد في التمييز بين فشل العقدة وبين مجرد وجود تأخير في الشبكة.
- العيوب: تتطلب مزامنة دقيقة للساعات بين جميع عُقد النظام (Clock Synchronization).

5. نبضات القلب مع التأكيد (Heartbeat with Acknowledgement)

- كيف تعمل: بعد استلام النبضة، يقوم المراقب بإرسال رسالة تأكيد (ACK) للطرف المرسل.
- المميزات: تضمن أن المسار بين الطرفين (إرسالاً واستقبالاً) يعمل بشكل سليم، وليس فقط العقدة.
- العيوب: تضاعف حجم الرسائل المتبادلة في الشبكة.

6. نبضات القلب مع النصاب (Heartbeat with Quorum)

- كيف تعمل: تُستخدم في البروتوكولات التي تتطلب إجماعاً (مثل Paxos أو Raft). تهدف النبضات إلى التأكد من وجود "نصاب" (أغلبية) من العقدة العاملة لاتخاذ القرارات.
- المميزات: تضمن سلامة واتساق البيانات في النظام.
- العيوب: صعوبة التنفيذ وتتطلب إدارة معقدة عند انضمام عقدة جديدة أو خروج عقدة قديمة من المجموعة.

الخلاصة: لا توجد آلية واحدة "مثالية"، فالاختيار يعتمد على طبيعة نظامك:

- إذا كنت تبحث عن البساطة، استخدم Push-Based.
- إذا كنت تريد تقليل الضغط على الشبكة، استخدم Pull-Based.
- إذا كنت تبني نظاماً حساساً يتطلب دقة عالية، اتجه نحو Health Check أو Quorum.

الفروقات بين الـ Reverse Proxy، و API Gateway، و Load Balancer

تُستخدم هذه الأدوات الثلاث لإدارة حركة البيانات في الأنظمة الحديثة، وكل واحدة تلعب دوراً مختلفاً "كأبطال خارقين" في إدارة الشبكات:

1. الوكيل العكسي (Reverse Proxy) - "مغير الهوية"

- الوظيفة الأساسية: يعمل كوسيط يخفي هوية وعنوان الخوادم الداخلية (Hide service address).
- أهميته: مثالي لجلب البيانات بشكل سري وحماية المواقع الحساسة من الهجمات الإلكترونية والمتطفلين.

2. بوابة واجهة برمجة التطبيقات (API Gateway) - "ساعي البريد"

- الوظيفة الأساسية: يعمل كنقطة دخول موحدة (Single entry point) تُنظم حركة الطلبات بين المستخدم ومجموعة متنوعة من الخدمات (Different APIs).
- أهميته: مثالي للتطبيقات المعقدة التي تحتوي على خدمات كثيرة تحتاج للتواصل مع بعضها البعض، حيث يقوم بتوصيل الطلبات للخدمة الصحيحة.

3. موازن الحمل (Load Balancer) - "شرطي المرور"

- الوظيفة الأساسية: يقوم بتوزيع حركة المرور (Traffic) بالتساوي على مجموعة من الخوادم التي تقدم نفس الخدمة (Same service).
- أهميته: ضروري للمواقع الشهيرة التي تشهد ضغطاً عالياً، حيث يمنع حدوث اختناقات في الخوادم (Bottlenecks) ويضمن استقرار الخدمة.

باختصار: متى تستخدم أيّاً منها؟

- استخدم **Reverse Proxy** إذا كان هدفك هو الخصوصية والتخفي.
- استخدم **API Gateway** إذا كان هدفك تنظيم الاتصالات وتوجيه الطلبات لخدمات متعددة.
- استخدم **Load Balancer** إذا كان هدفك التحكم في حركة المرور وتوزيع الضغط على الخوادم.

في كثير من الأحيان، يمكن استخدام هذه الأدوات الثلاث معاً لتشكل فريقاً متكاملًا يحافظ على أمان وكفاءة نظامك الرقمي.

أهم 6 أنماط لإدارة البيانات (Data Management Patterns)

تُستخدم هذه الأنماط لتحسين أداء الأنظمة البرمجية، وتسهيل التعامل مع البيانات الضخمة والمعقدة:

1. الكاش الجانبي (Cache Aside)

- **الوظيفة:** عندما يحتاج التطبيق لبيانات، يبحث أولاً في الـ "Cache". إذا لم يجدها، يجلبها من قاعدة البيانات ويخزنها في الـ "Cache" لاستخدامها لاحقاً.
- **الهدف:** تحسين سرعة الوصول للبيانات التي يتم قراءتها بكثرة.

2. العرض المُحقق (Materialized View)

- **الوظيفة:** هو كائن في قاعدة البيانات يحتوي على نتائج استعلام معقد (حسابات أو تجميعات) مخزنة فعلياً على القرص.
- **الهدف:** تسريع الاستعلامات المعقدة بدلاً من إعادة حسابها في كل مرة.

3. فصل أوامر القراءة والكتابة (CQRS)

- **الوظيفة:** نمط معماري يفصل بين النماذج المستخدمة لتحديث البيانات (الكتابة) والنماذج المستخدمة للاستعلام عنها (القراءة).
- **الهدف:** تحسين الأداء والقابلية للتوسع (Scalability) بشكل مستقل لكل عملية.

4. مصادر الأحداث (Event Sourcing)

- **الوظيفة:** بدلاً من تخزين الحالة الحالية للبيانات فقط، يتم تخزين "سجل كامل" لكل التغييرات (الأحداث) التي حدثت عبر الزمن.
- **الهدف:** توفير سجل تدقيق (Audit trail) كامل، والقدرة على إعادة بناء الحالات السابقة أو التراجع عن العمليات.

5. جدول الفهرسة (Index Table)

- **الوظيفة:** إنشاء جداول إضافية (فهارس ثانوية) مصممة خصيصاً لتسريع عمليات بحث معينة.
- **الهدف:** تسريع جلب البيانات دون الحاجة لمسح قاعدة البيانات بالكامل.

6. التجزئة (Sharding)

- **الوظيفة:** تقسيم البيانات إلى أجزاء أصغر (Shards) وتوزيعها على خوادم قاعدة بيانات متعددة.
- **الهدف:** تحسين الأداء من خلال التوسع الأفقي (Horizontal Scaling) والتعامل مع كميات هائلة من البيانات والمستخدمين.

ملخص سريع:

- للسرعة العادية: **Cache Aside**.
- للاستعلامات المعقدة المتكررة: **Materialized View**.
- للتوسع الكبير: **CQRS** أو **Sharding**.
- للتوثيق والمراجعة: **Event Sourcing**.
- للبحث السريع: **Index Table**.

أهم 4 خوارزميات لتقسيم البيانات (Data Sharding Algorithms)

تُستخدم هذه الخوارزميات لتقسيم البيانات الضخمة إلى أجزاء أصغر وأكثر قابلية للإدارة تُسمى "Shards" لتعزيز أداء الأنظمة:

1. التقسيم القائم على النطاق (Range-Based Sharding)

- **آلية العمل:** يتم تقسيم البيانات بناءً على نطاق محدد من القيم (مثل الترتيب الأبجدي للأسماء أو فترات زمنية).
- **المميزات:** سهولة التنفيذ.
- **العيوب:** قد تؤدي إلى توزيع غير متساوٍ للبيانات (بعض العقد قد تكون مشغولة جداً بينما الأخرى خاملة).

2. التقسيم القائم على التجزئة (Hash-Based Sharding)

- **آلية العمل:** يتم تطبيق دالة التجزئة (Hash Function) على مفتاح معين من البيانات (مثل معرف المستخدم أو المعاملة).
- **المميزات:** تضمن توزيعاً أكثر توازناً للبيانات عبر العقد.
- **العيوب:** تتطلب اختيار دالة "Hash" دقيقة لتجنب التصادم (Hash Collision).

3. التجزئة المتسقة (Consistent Hashing)

- **آلية العمل:** تطوير للتقسيم القائم على التجزئة، حيث يتم تمثيل البيانات والعقد على حلقة دائرية.
- **المميزات:** تقل بشكل كبير من كمية البيانات التي تحتاج للنقل عند إضافة أو إزالة عقد جديدة.
- **العيوب:** قد تزداد التعقيد عند إدارة عدد كبير جداً من العقد الافتراضية.

4. التجزئة عبر الصناديق الافتراضية (Virtual Bucket Hashing)

- **آلية العمل:** يتم تعيين البيانات إلى "صناديق منطقية" (Logical Buckets) أولاً، ثم يتم ربط هذه الصناديق بالـ "Shards" الفيزيائية.
- **المميزات:** توفر مرونة عالية جداً في إدارة وإعادة توازن البيانات دون الحاجة لنقل كميات كبيرة منها.
- **العيوب:** تتطلب إدارة إضافية لعملية الربط (Mapping) بين الصناديق الافتراضية والتخزين الفيزيائي.

باختصار:

- إذا كنت تحتاج للبساطة وتتعامل مع نطاقات واضحة: استخدم **Range-Based**.
- إذا كنت تريد توازناً جيداً للبيانات: استخدم **Hash-Based**.
- إذا كان نظامك يتوسع (إضافة/إزالة عقد) بشكل متكرر: استخدم **Consistent Hashing** أو **Virtual Bucket Hashing**.

آلية عمل تقنية gRPC

مفهوم الـ (RPC Remote Procedure Call)

يُطلق عليه "استدعاء الإجراء عن بُعد" لأنه يُمكن الخدمات البرمجية المستضافة على خوادم مختلفة ومتباعدة من التواصل فيما بينها ضمن معمارية الخدمات المصغرة (Microservices). ومن وجهة نظر المستخدم أو المطور، فإن هذا الاتصال يظهر ويعمل وكأنه استدعاء لدالة محلية (Local Function Call) داخل نفس الجهاز، بالرغم من أنه يتم بين خادمين منفصلين (مثل انتقال الطلب من خادم خدمة الطلبات Server A إلى خادم خدمة الدفع Server B).

لماذا يعتبر gRPC أسرع بـ 5 مرات من نظام JSON التقليدي؟

يعود الفضل في هذه السرعة العالية إلى عاملين رئيسيين يظهران في الرسم التوضيحي:

1. **التشفير الثنائي (ProtoBuf):** بدلاً من إرسال البيانات بصيغة نصية واضحة وحجم كبير مثل JSON، يقوم gRPC بتحويل البيانات وتعبئتها في صيغة ثنائية (Binary Encoding) مضغوطة وصغيرة الحجم.
2. **بروتوكول HTTP/2:** تُرسل هذه الحزم عبر الشبكة باستخدام بروتوكول HTTP/2 الذي يحتوي على تحسينات برمجية فائقة لنقل البيانات.

خطوات تدفق البيانات بالتفصيل (gRPC Overall Flow)

تنقسم دورة حياة الطلب والاستجابة في gRPC إلى المراحل التالية كما تبينها صورتي [image_038e8d.png](#) و [image_038eaa.png](#):

- **الخطوة 1 (بدء الطلب):** يقوم تطبيق العميل (Client App) بإرسال طلب REST تقليدي، وعادة ما يكون نص الطلب بصيغة JSON.
- **الخطوات 2 - 4 (تجهيز الطلب عند العميل):** تستقبل خدمة الطلبات (gRPC client - Order Service) طلب الـ REST، وتقوم بتحويله وإجراء استدعاء RPC. هنا يتولى نظام gRPC تشفير الطلب (Client Stub) وتحويله من صيغة نصية إلى صيغة ثنائية (Binary format) ثم يمرره إلى طبقة النقل السفلى.
- **الخطوة 5 (النقل عبر الشبكة):** يقوم gRPC بإرسال الحزم المشفرة عبر الشبكة مستخدماً بروتوكول HTTP/2.
- **الخطوات 6 - 8 (معالجة الطلب عند الخادم):** تستقبل خدمة الدفع (gRPC server - Payment Service) الحزم القادمة من الشبكة، وتقوم بفك تشفيرها (Decoding)، ثم تستدعي تطبيق الخادم محلياً لمعالجة العملية.
- **الخطوات 9 - 11 (إرسال الرد من الخادم):** يعيد تطبيق الخادم النتيجة، وتُشفّر مجدداً إلى صيغة ثنائية، ثم تُرسل عبر طبقة النقل إلى الشبكة متوجهة إلى العميل.
- **الخطوات 12 - 14 (استلام الرد وعرضه):** تستقبل خدمة الطلبات (Order Service) الحزم المرتدة عبر الشبكة، وتُفك تشفيرها، ومن ثم تُرسل النتيجة النهائية الواضحة إلى تطبيق العميل.

أهم 9 ممارسات لبناء أنظمة الخدمات المصغرة (Microservices):

بناء نظام باستخدام الخدمات المصغرة عملية معقدة للغاية، ولضمان نجاحها يجب اتباع هذه المبادئ الأساسية:

1. **التصميم مع وضع الفشل في الاعتبار (Design for Failure):** بما أن الفشل في الأنظمة الموزعة أمر حتمي، يجب تصميم النظام ليتحمل الأعطال على مستويات مختلفة (البنية التحتية، قاعدة البيانات، والخدمات الفردية) باستخدام تقنيات مثل قواطع الدائرة (Circuit Breakers).
2. **بناء خدمات صغيرة (Build Small Services):** يجب أن تقوم الخدمة الواحدة بمهمة محددة وتؤديها بشكل ممتاز، وتجنب جعل الخدمة الواحدة تقوم بمهام متعددة في آن واحد.

3. استخدام بروتوكولات خفيفة الوزن (**Lightweight Protocols**): التواصل هو جوهر الأنظمة الموزعة، لذا يجب استخدام بروتوكولات خفيفة وسريعة للتواصل بين الخدمات، مثل REST أو gRPC أو وسطاء الرسائل (Message Brokers).
4. تطبيق اكتشاف الخدمات (**Implement Service Discovery**): تحتاج الخدمات إلى العثور على بعضها البعض عبر الشبكة ديناميكياً، ويمكن تحقيق ذلك باستخدام أدوات مثل Consul أو Eureka أو Kubernetes Services.
5. ملكية البيانات (**Data Ownership**): يجب أن تكون كل خدمة مسؤولة عن بياناتها الخاصة وإدارتها بشكل مستقل، لتقليل الاعتماد المتبادل بين الخدمات والسماح لها بالتطور بشكل منفصل.
6. استخدام أنماط المرونة (**Use Resiliency Patterns**): تحسين توافر الخدمات من خلال تطبيق أنماط محددة مثل سياسات إعادة المحاولة (Retry policies)، التخزين المؤقت (Caching)، وتحديد معدل الطلبات (Rate limiting).
7. الأمان على جميع المستويات (**Security at all levels**): بما أن مساحة الهجوم في أنظمة الخدمات المصغرة واسعة، يجب تأمين كل مستوى من مسارات التواصل بين الخدمات.
8. التسجيل المركزي للسجلات (**Centralized Logging**): مع وجود خدمات متعددة، يصبح تتبع المشكلات أمراً بالغ الصعوبة، لذا فإن تجميع السجلات (Logs) في مكان واحد هو أمر حيوي لاكتشاف الأخطاء.
9. استخدام تقنيات الحاويات (**Containerization**): استخدام أدوات مثل Docker و Kubernetes لنشر الخدمات بشكل معزول، مما يبسط عمليات التوسع (Scaling) والنشر.

احتمالية فقدان الرسائل في نظام Kafka وكيفية حدوث ذلك في مراحل مختلفة

خلافًا للاعتقاد الشائع بأن Kafka يضمن عدم ضياع الرسائل بتصميمه الأساسي، فمن الممكن حدوث ذلك بناءً على كيفية إعداد وتكوين النظام في ثلاث مراحل رئيسية:

1. مرحلة المُنتج (Producer)

- عند استخدام الإرسال غير المتزامن (`producer.send()`)، لا يتم إرسال الرسالة مباشرة إلى الوسيط (Broker).
- تتضمن العملية خيوط معالجة (Threads) ومراكز سجلات (Record accumulator)، وإذا لم يتم تكوين إعدادات الـ `acks` (الإقرارات) والـ `retries` (إعادة المحاولة) بشكل صحيح، فقد تضيع الرسائل.

2. مرحلة الوسيط (Broker)

- على الرغم من أن نظام الـ Broker لا يفقد الرسائل أثناء العمل الطبيعي، إلا أن هناك حالات قصوى قد تؤدي لفقدان:
- التفريغ غير المتزامن للقرص: تُفرغ الرسائل عادةً إلى القرص بشكل غير متزامن لتحسين سرعة الإدخال/الإخراج؛ فإذا تعطل النظام قبل حدوث هذا التفريغ، ستضيع الرسائل.
- نسخ البيانات (Replication): إذا لم يتم تكوين النسخ المتماثلة في عنقود (Kafka Cluster) بشكل صحيح للحفاظ على نسخة صالحة من البيانات، فقد يحدث فقدان.

3. مرحلة المستهلك (Consumer)

- الالتزام التلقائي (Auto-commit): قد يقوم هذا الإعداد بتأكيد معالجة السجلات قبل إتمام معالجتها فعلياً؛ فإذا تعطل المستهلك أثناء المعالجة، فقد لا تُعالج بعض السجلات أبداً.

- أفضل ممارسة للالتزام: يُنصح بالجمع بين الالتزام المتزامن وغير المتزامن؛ حيث يُستخدم الالتزام غير المتزامن داخل حلقة المعالجة لتحقيق إنتاجية أعلى، بينما يُستخدم الالتزام المتزامن عند معالجة الاستثناءات (Exception handling) لضمان التزام آخر إزاحة (Offset) دائماً.

أهم 5 حالات استخدام لنظام Kafka

يتميز Kafka بقدرته على معالجة كميات ضخمة من البيانات، حيث يحتفظ بالرسائل حتى انتهاء صلاحيتها ويسمح للمستهلكين بسحب البيانات حسب سرعته الخاصة.

إليك حالات الاستخدام:

- **تحليل السجلات (Log Analysis):** يُستخدم لمعالجة وتحليل سجلات الأحداث الضخمة القادمة من خدمات مختلفة، وغالباً ما يتم ذلك بالتعاون مع أدوات مثل Elastic و Kibana.
- **بث البيانات في أنظمة التوصيات (Data Streaming in Recommendations):** يعمل على بث بيانات تفاعل المستخدمين (مثل النقرات) لتحليلها وتغذية نماذج تعلم الآلة (Machine Learning) لتقديم توصيات مخصصة.
- **مراقبة الأنظمة والتنبيهات (System Monitoring and Alerting):** يُستخدم لمراقبة حالة الخدمات لحظياً وإطلاق تنبيهات عند حدوث أي خلل.
- **التقاط تغيير البيانات (Change Data Capture - CDC):** يقوم بالتقاط التغييرات التي تحدث في قواعد البيانات المصدر (Transaction logs) ونقلها إلى أنظمة أخرى أو قواعد بيانات متماثلة (Replica Databases).
- **ترحيل الأنظمة (System Migration):** يسهل عملية الانتقال من إصدار قديم للخدمة إلى إصدار جديد من خلال مقارنة النتائج بين النظامين القديم والجديد (Reconciliation) لضمان سلامة البيانات.

"الأربعة الرائعون" (The Fantastic Four) في تصميم الأنظمة

1. القابلية للتوسع (Scalability)

- **الهدف:** ضمان قدرة النظام على التعامل مع أحمال متزايدة دون التأثير على الأداء.
- **طرق التنفيذ:**
 - التوسع الرأسي (Vertical Scaling): زيادة موارد الخادم الواحد (مثل المعالج أو الذاكرة).
 - التوسع الأفقي (Horizontal Scaling): إضافة المزيد من الخوادم.
 - الخدمات المصغرة (Microservices): تقسيم النظام لخدمات مستقلة ليسهل توسيع كل جزء على حدة.

2. التوافرية (Availability)

- **الهدف:** ضمان بقاء النظام جاهزاً لخدمة المستخدمين دائماً مع تقليل وقت التوقف إلى أدنى حد ممكن.
- **طرق التنفيذ:**
 - موازنة الحمل (Load Balancing): توزيع الطلبات على خوادم متعددة لتجنب الضغط على خادم واحد.
 - النسخ المتماثل (Replication): الاحتفاظ بنسخ من البيانات لضمان عدم فقدانها.
 - آليات الفشل (Failover): التحويل التلقائي لنظام بديل عند حدوث عطل في النظام الأساسي.

3. الموثوقية (Reliability)

- **الهدف:** ضمان أن النظام يعمل بشكل ثابت ومستمر ويقدم نتائج صحيحة في كل مرة.

- طرق التنفيذ:
 - المراقبة وتسجيل البيانات (Monitoring & Logging): استخدام أدوات مثل Logstash وElasticsearch وKibana لتتبع النظام واكتشاف المشاكل.
 - معالجة الأخطاء (Error Handling): تطبيق آليات مثل "إعادة المحاولة" (Retry) عند فشل الطلبات.
 - الاختبارات المؤتمتة (Automated Tests): التأكد من جودة الكود قبل طرحه.

4. الأداء (Performance)

- الهدف: قدرة النظام على تنفيذ المهام المطلوبة بالمعدل المتوقع حتى تحت أقصى أحمال التشغيل وباستخدام الموارد المتاحة.
- طرق التنفيذ:
 - فهرسة قاعدة البيانات (Database Indexing): لتسريع عمليات البحث واسترجاع البيانات.
 - التخزين المؤقت (Caching): حفظ البيانات المستخدمة بكثرة لسرعة الوصول إليها.
 - المعالجة غير المتزامنة (Async Processing): تنفيذ المهام في الخلفية لعدم إعاقة استجابة النظام للمستخدم.

باختصار، يمثل هؤلاء "الأربعة الرائعون" الأعمدة التي يقوم عليها بناء الأنظمة البرمجية الناجحة.

أهم المجلدات في نظام ملفات لينكس (Linux File Systems)

يوضح هذا الملف التنظيم الهيكلي الذي وضعه معيار التسلسل الهرمي لنظام الملفات (FHS) في عام 1994 لضمان اتساق النظام:

المجلدات الأساسية ووظائفها:

- **/bin/**: يحتوي على الملفات التنفيذية للأوامر الأساسية للنظام.
- **/boot/**: يحتوي على الملفات اللازمة لتحميل وتشغيل نظام التشغيل (System boot loader files).
- **/dev/**: يحتوي على ملفات الأجهزة (Device files).
- **/etc/**: يحتوي على ملفات الإعدادات الخاصة بالنظام (Host-specific configuration files).
- **/home/**: المجلد الخاص بمجلدات المستخدمين الشخصية (User home directory).
- **/lib/**: يحتوي على وحدات المكتبات المشتركة (Shared library modules).
- **/media/**: مخصص لملفات الوسائط القابلة للإزالة مثل أقراص CD-ROM.
- **/mnt/**: يُستخدم لتركيب (mount) أنظمة الملفات المؤقتة.
- **/opt/**: مخصص لحزم البرامج الإضافية (Add-on application software packages).
- **/proc/**: مجلد افتراضي يوفر واجهة لهياكل بيانات النواة (Kernel data structures).
- **/root/**: المجلد الرئيسي للمستخدم الجذر (Home directory for root user).
- **/run/**: يحتوي على البيانات الخاصة بالبرامج التي تعمل حالياً (Run-time program data).
- **/sbin/**: يحتوي على الملفات التنفيذية الخاصة بمدير النظام (System binaries).
- **/srv/**: يحتوي على بيانات خاصة بالخدمات التي يقدمها النظام.
- **/sys/**: مجلد افتراضي يوفر معلومات حول مكونات النظام.
- **/tmp/**: مخصص للملفات المؤقتة (Temporary files).
- **/usr/**: يحتوي على الموارد الأساسية لنظام يونكس (Unix System Resources).
- **/var/**: يحتوي على الملفات التي يتوقع أن تتغير محتوياتها باستمرار (مثل السجلات).

ملاحظة هامة: على الرغم من وجود معيار FHS الذي يضمن وجود هيكل متسق عبر توزيعات لينكس المختلفة، إلا أن العديد من التوزيعات قد لا تلتزم به بشكل صارم، حيث تقوم بدمج عناصرها الفريدة أو تعديلها لتلبية متطلبات خاصة.

أهم 6 استراتيجيات لتقليل وقت الاستجابة (Latency) في الأنظمة البرمجية

تُعد سرعة استجابة النظام أمراً حيوياً، حيث اكتشفت أمازون سابقاً أن كل تأخير بمقدار 100 مللي ثانية يكلفها 1% من المبيعات، وهو ما يعادل مليارات الدولارات حالياً.

إليك الاستراتيجيات المتبعة لتقليل هذا التأخير:

- **فهرسة قاعدة البيانات (Database Indexing):**
 - يتم ذلك بإنشاء الفهارس المناسبة للبيانات.
 - يجب تحسين وإعادة صياغة الاستعلامات البطيئة في قاعدة البيانات.
- **التخزين المؤقت (Caching):**
 - يتم تخزين البيانات التي يتم الوصول إليها بكثرة في "ذاكرة مؤقتة" (Cache).
 - يساهم ذلك في تقليل الحاجة لعمليات البحث المكلفة في قاعدة البيانات.
- **موازنة الحمل (Load Balancing):**
 - توزيع الطلبات بشكل متساوٍ بين خوادم متعددة.
 - استخدام النوع المناسب من موازنات الحمل مع الخوارزمية الملائمة.
- **شبكات توصيل المحتوى (Content Delivery Networks - CDN):**
 - تخزين المحتوى الثابت بالقرب من المستخدمين النهائيين.
 - تقليل المسافة الجغرافية لخفض وقت الاستجابة.
- **المعالجة غير المتزامنة (Async Processing):**
 - عدم حظر مسار الطلب الرئيسي عند القيام بمهام طويلة.
 - تنفيذ هذه المهام في الخلفية.
- **ضغط البيانات (Data Compression):**
 - ضغط البيانات قبل إرسالها عبر الشبكة.
 - استخدام مكتبات متخصصة مثل **gzip** و **zlib** للضغط.

هذه المفاهيم هي الركائز الأساسية في هندسة الأنظمة الموزعة (Distributed Systems) ضرورية لتصميم أنظمة عالية التوافر (High Availability)

1. Byzantine Fault (الخطأ البيزنطي)

هو السيناريو الأكثر تعقيداً في الأنظمة الموزعة، حيث لا تكتفي العقدة (Node) بالتوقف عن العمل، بل تتصرف بشكل مضلل أو خبيث.

- **المفهوم:** قد ترسل العقدة معلومات متناقضة لأجزاء مختلفة من النظام، مما يجعل من الصعب على بقية العقد اتخاذ قرار موحد.
- **مثال:** في نظام مالي، قد ترسل عقدة رسالة "تم خصم الرصيد" للخدمة (أ)، ورسالة "لم يتم خصم الرصيد" للخدمة (ب). الأنظمة التي تتحمل هذا الخطأ تسمى (Byzantine Fault Tolerant - BFT).

2. Bottleneck (عنق الزجاجة)

هو أي نقطة في النظام الموزع تكون فيها السعة (Capacity) أو سرعة المعالجة أقل من بقية أجزاء النظام، مما يؤدي إلى تباطؤ النظام بأكمله.

- **المفهوم:** حتى لو كانت باقي الخدمات سريعة جداً، فإن النظام لن يعمل أسرع من أبطأ جزء فيه.
- **مثال:** إذا كان لديك 100 ألف جهاز IoT يرسلون بيانات، وقاعدة البيانات لا تستطيع استقبال أكثر من 20 ألف سجل في الثانية، فإن قاعدة البيانات هنا هي "عنق الزجاجة".

3. Poison Pill (الرسالة السامة)

هي رسالة (Message) أو طلب (Request) يحتوي على بيانات تالفة أو غير متوقعة، تؤدي عند معالجتها من قبل الـ Consumer إلى انهيار الخدمة أو حدوث استثناءات (Exceptions) متكررة.

- **المفهوم:** تُسمى "سامة" لأنها تستهلك موارد النظام وتمنع الخدمة من معالجة الرسائل السليمة التالية.
- **الحل:** يتم عزل هذه الرسائل في طابور خاص يسمى (Dead Letter Exchange - DLX) لمنعها من إيقاف سير العمل.

4. Split-Brain (انقسام الدماغ)

تحدث هذه المشكلة في الأنظمة التي تعتمد على التكرار (Replication) أو العناقيد (Clusters) عند حدوث انقطاع في الشبكة (Network Partition).

- **المفهوم:** ينقسم النظام إلى جزأين معزولين، حيث يعتقد كل جزء أنه "القائد" (Leader) الوحيد، فيبدأ كلا الجزأين باتخاذ قرارات مستقلة ومتضاربة على نفس البيانات.
- **الخطر:** يؤدي هذا إلى تعارض خطير في البيانات (Data Inconsistency) يصعب حله لاحقاً.

5. Phoenix Pattern / Immutable Image (نمط العنقاء)

بدلاً من محاولة "إصلاح" نظام متضرر أو عالق في حالة سيئة، نقوم بتدميره تماماً وإعادة بنائه من صورة (Image) أصلية سليمة ومجمدة (Immutable).

- **المفهوم:** مثل أسطورة طائر العنقاء الذي يولد من رماده. في بيئات الـ Cloud Native، إذا تعطلت حاوية (Container) أو خادم، لا نقوم بعمل SSH لإصلاح الخطأ، بل نقوم بإنهاء الحاوية الحالية وتشغيل نسخة جديدة تماماً منها.
- **الفائدة:** هذا يضمن أن البيئة دائماً "نظيفة" ولا تعاني من تراكم الأخطاء أو التغييرات الجانبية (Side Effects) التي قد تحدث أثناء تشغيل النظام لفترات طويلة.

حل النموذج المغلق 1

السؤال الأول: تحديات الـ Scaling (بيتزا)

- ما هو حل مشكلة الخادم البطيء في الـ Round-Robin؟ ولماذا قد لا يكفي Least Connections؟

- الحل هو استخدام **Least Connections** لتوجيه الطلبات للخادم الأقل ضغطاً.
- قد لا يكفي هذا الحل لأن عدد الاتصالات لا يعبر دائماً عن حجم المعالجة المطلوبة؛ فقد يكون خادم لديه اتصالات قليلة لكنها ذات طلبات ثقيلة جداً، لذا يفضل استخدام موازنة حمل ديناميكية (Dynamic Load Balancing) تعتمد على استهلاك الموارد (CPU/RAM).
- ماذا يحدث إذا فشل الـ **Outbox Pattern**، وكيف يدخل **RabbitMQ** في الصورة؟
 - نستخدم الـ **Outbox Pattern** بحفظ الرسالة في جدول خاص داخل قاعدة البيانات ضمن نفس المعاملة (Transaction) لضمان عدم ضياعها.
 - يعمل **RabbitMQ** كـ "ناقل رسائل" يستلم البيانات من الـ **Outbox Publisher** بعد نجاح العملية في قاعدة البيانات.
- كيف تمنع خصم 7 بيتزات من حساب "أحمد" عند ضغطه الزر 7 مرات؟
 - نطبق آلية الـ **Idempotency**؛ حيث نرفق **Idempotency-Key** (مثل UUID) مع كل طلب دفع، ويقوم الخادم بالتحقق إذا عولج هذا المفتاح سابقاً، فإذا وجد يرفض الخصم المتكرر.

السؤال الثاني: اللعبة العالمية (Latency & Consistency)

- كيف تستخدم **CDN** و **Server-in-Mesh** لتقليل الـ **Latency**؟ ومتى تختار توحيد اللاعبين على خادم واحد؟
 - نستخدم **CDN** لتخزين المحتوى الساكن قريباً من اللاعب، و **Server-in-Mesh** لتقليل المسافة المادية بين اللاعب والخادم.
 - نختار توحيد اللاعبين على خادم واحد عندما تتطلب اللعبة **Strong Consistency** (اتساقاً قوياً) لمنع التلاعب بالنتائج.
- تحديث متزامن (**Synchronous**) أم لا متزامن (**Asynchronous**)؟ وما الثمن؟
 - المتزامن يضمن اتساق البيانات لكنه يسبب تأخيراً (**High Latency**).
 - اللا متزامن سريع جداً لكنه يعاني من مخاطرة عدم اتساق البيانات (**Data Inconsistency**) عند تعطل الخادم الرئيسي.
- الفرق بين **Liveness vs Readiness** لاكتشاف تعطل الخوادم؟
 - **Liveness**: هو فحص "هل الخادم حي؟" وإذا تعطل يتم إعادة تشغيله.
 - **Readiness**: هو فحص "هل الخادم جاهز للمعالجة؟" (مثلاً للتأكد من انتهاء تحميل البيانات).

السؤال الثالث: Black Friday (المرونة - Resilience)

- كيف تتخذ الموقف بـ **Circuit Breaker** و **Fallback** وما هي الخطة B؟
 - **Circuit Breaker**: يقطع الاتصال بالخدمة البطيئة فوراً لمنع انهيار باقي النظام.
 - الخطة B (**Fallback**): عرض منتجات افتراضية أو ثابتة للمستخدم بدلاً من شاشة فارغة.
- لماذا الـ **Retry** بدون **Backoff** قد يقتل خدمة الدفع؟
 - لأنه يسبب "Thundering Herd Problem"؛ حيث تقوم جميع الخوادم بإعادة إرسال الطلبات في وقت واحد مما يؤدي لانهايار الخدمة نهائياً.
- كيف تستخدم **Consistent Hashing** وماذا يحدث لو تعطل خادم **Cache**؟
 - يوزع الـ **Keys** على الخوادم بشكل توزيعي.
 - إذا تعطل خادم واحد، لا ينهار النظام؛ فقط البيانات الموجودة على هذا الخادم المحدد تُفقد وتحديث حالة **Cache Miss** للطلبات الخاصة به فقط.

السؤال الرابع: تطبيق الدردشة

- متى يكون **Eventual Consistency** مقبولاً في الدردشة ومتى يكون كارثة؟
 - مقبول في الدردشة لأن تأخر الرسالة لثوانٍ لا يضر بتجربة المستخدم.
 - كارثة في الأنظمة المالية أو التذاكر حيث يجب أن يكون الرصيد أو الحجز دقيقاً فوراً.
- أين تذهب الرسائل الفاشلة وكيف تتعامل مع **DLX**؟
 - تذهب لـ **(Dead Letter Exchange (DLX**.

- لا نتجاهلها، بل نستخدم Worker لإعادة المحاولة (Retry) بعد وقت معين أو لتتبيه المستخدم.
- كيف يعرف الخادم أن زميله تعطل بدون خادم مركزي؟ (Gossip Protocol)
 - كل خادم يتبادل حالته مع "جيرانه" عشوائياً، وينتشر الخبر بسرعة في الشبكة كـ "إشاعة" حتى يعلم الجميع بتعطل الخادم المتوقف دون الحاجة لمرجع مركزي.

حل النموذج المغلق 2

السؤال الثاني: مفاهيم JavaRMI

1. في RMI، أرسلت كائن Order (يحتوي List) من Client إلى Server وعدلت quantity في الـ Server، هل يرى الـ Client التعديل؟ ولماذا؟
 - الإجابة: لا يرى الـ Client التعديل. السبب هو أن كائنات JavaRMI تُرسل **by Value** (عن طريق Serialization)، مما يعني أن الـ Server يستلم نسخة (Copy) من الكائن وليس مرجعاً (Reference) للكائن الأصلي في ذاكرة الـ Client.
2. إذا عدلت على كائن Remote عبر RMI، هل سترى التغييرات في نفس الـ JVM التي أنشأت الكائن الأصلي؟
 - الإجابة: نعم، سيتم رؤية التغييرات.

السبب:

- في Java RMI، عندما يتم تمرير كائن يمتد من **Remote** (ويتم الوصول إليه عبر Stub)، فإننا لا نقوم بنسخ الكائن، بل نرسل مرجعاً (Reference) أو وكيل (Stub) يشير إلى الكائن الموجود في الـ JVM الأصلية.
- في هذه الحالة، لا يتم عمل Serialization لبينات الكائن نفسه، بل يتم توجيه استدعاءات الدوال (Method Invocations) عبر الشبكة لتُنفذ على الكائن الأصلي في الـ JVM التي أنشأته.
- بما أن الطرفين يشيران إلى نفس الكائن (أو يتحكمان في نفس الكائن الأصلي عبر الـ Stub)، فإن أي تعديل يتم استدعاؤه سيؤثر مباشرة على الكائن في مكانه الأصلي، وبالتالي ستعكس التغييرات لكل من يملك مرجعاً (Reference) لهذا الكائن Remote.

السؤال الثالث: أسئلة تقنية متنوعة

1. كيف تضبط NGINX للتعامل مع سيرفر أبطأ بـ 10 مرات؟
 - الإجابة: نستخدم خاصية **weight** في إعدادات NGINX لتحديد حصة الخادم البطيء من الطلبات بحيث تكون أقل من غيره، أو نستخدم **least_conn** ليتم توجيه الطلبات للخادم الذي يمتلك حالياً أقل عدد من الاتصالات النشطة.
2. كيف تشخص بطء الطلبات (15 ثانية بدل 200ms) في نظام **Microservices** سليم؟
 - الإجابة: نستخدم تقنيات التتبع الموزع (Distributed Tracing) مثل Jaeger أو Zipkin لتحديد الخدمة التي تسبب التأخير في سلسلة الاستدعاءات، ونفحص احتمالية وجود اختناق في الموارد (Resource Contention) أو مشاكل في الـ Thread Pool.
3. لماذا يستخدم **Cassandra** بروتوكول **Gossip** بدلاً من **Heartbeat** مركزي؟ وما هو الـ **Cost** الخفي؟
 - الإجابة: **Gossip** بروتوكول لامركزي يزيل نقطة الفشل الواحدة (Single Point of Failure). التكلفة الخفية هي استهلاك جزء من عرض النطاق الترددي (Bandwidth) لنشر حالة العقد في المجموعات الكبيرة (مثل 1000 عقدة).
4. كيف تنفذ **Delayed Job Queue** لمدة 5 دقائق باستخدام **TTL + DLX**؟
 - الإجابة: نرسل المهمة إلى طابور (Queue) تم ضبطه بـ **TTL** يساوي 5 دقائق وبدون أي Consumer، ثم نربط هذا الطابور بـ **Dead Letter Exchange (DLX)** ليتم نقل المهمة تلقائياً للطابور الوجهة بعد انتهاء الوقت.

5. تأثير (50ms Latency) على عملية (100ms)؟ وهل ستنتم المعالجة بالتوازي لـ 1000 عميل؟
 ○ الإجابة: ستستغرق العملية 150ms إجمالاً. نعم، ستنتم المعالجة بالتوازي إذا كانت الخدمة مصممة لدعم تعدد الخيوط (Multithreading) في الـ Remote Object.
6. لماذا يفشل RMI أحياناً بـ **ClassNotFoundException** رغم وجود الكلاس في الـ **Classpath**؟
 ○ الإجابة: يحدث ذلك غالباً بسبب عدم توافق إصدار الـ **SerialVersionUID** بين الـ Client والـ Server، أو لأن الـ Server يفتقر إلى المكتبات الاعتمادية (Dependencies) الخاصة بالكائن المرسل.
7. كم طلباً فعلياً سيصل إلى خدمة B المعطلة قبل فتح الـ **Circuit Breaker**؟
 ○ الإجابة: سيصل 15 طلباً؛ حيث أن الـ **failureThreshold** هو 5، ووجود 3 محاولات (Retries) لكل طلب يجعل النظام يجرب $5 \times 3 = 15$ مرة قبل أن يفتح الـ **Circuit Breaker**.
8. لماذا التسلسل **A,A,A,A,A,B,C** كارثي في **Weighted Round Robin**؟
 ○ الإجابة: لأنه يسبب تجمع الطلبات (Burst) على الخادم A بشكل متتالي مما يسبب ضغطاً مفاجئاً (Hotspot). الحل هو استخدام خوارزمية **Smooth Weighted Round Robin** التي توازن توزيع الطلبات عبر الزمن بشكل أكثر انتظاماً.

السؤال الرابع: نقد ادعاءات المهندسين

- الخطأ في النقاط:
 - **Linearizable + Available**: مستحيل حسب نظرية CAP؛ يجب التضحية بأحدهما أثناء انقطاع الشبكة.
 - **Zero-latency**: مستحيل فيزيائياً بسبب قانون سرعة الضوء والمسافات الجغرافية عبر 5 قارات.
 - **No consensus protocol**: مستحيل لنظام موزع يحتاج للتوافق (Consistency) في حال وجود أعطال.

السؤال الخامس: الجيران والياتسب

- البروتوكول: Gossip Protocol. يعمل بشكل أسي $(O(\log n))$.
- الجار الكاذب (Byzantine Fault): يسبب نشر معلومات مضللة، وللتعامل معه نحتاج خوارزميات إجماع متقدمة (مثل BFT) للتحقق من صحة الخبر.

السؤال السادس: Exponential Backoff و Circuit Breaker

- الحل: نطبق **Exponential Backoff** بزيادة زمن الانتظار تدريجياً بين كل محاولة إعادة اتصال (Retry) لتقليل الضغط. ونطبق **Circuit Breaker** ليقوم بفتح الدارة (Open State) وإيقاف أي محاولات اتصال للخدمة المعطلة لفترة زمنية محددة حتى تتعافى الخدمة.

السؤال السابع: معضلة واتساب

- الترتيب: يستخدم واتساب ترتيباً زمنياً منطقياً (Logical Timestamps) لترتيب الرسائل.
- **Eventual Consistency**: يعتبر الواتساب نظام **Eventual Consistency** لأنه يعطي الأولوية لتوفر الخدمة (Availability) وتجربة المستخدم السلسة، مع ضمان تماثل البيانات بين جميع الأطراف بعد فترة زمنية قصيرة.
- منع **Split-Brain**: يتم حلها باستخدام خوارزميات الإجماع أو طوابع الوقت المنطقية لضمان ترتيب الرسائل بشكل صحيح رغم تأخر وصول بعضها.

السؤال التاسع: بوفيه السوشي

- الحل: نستخدم **Rate Limiting** عند مدخل البوفيه للتحكم في عدد الزبائن الداخلين. ونستخدم **Jitter** (إضافة تأخير عشوائي بسيط للزبائن) لكسر تزامنهم ومنع مشكلة الـ (Thundering Herd) عند طبق السوشي.

السؤال العاشر: الفراغات

1. **Idempotency**: أنا الذي إذا ضغطت علي مئة مرة، لن يتغير شيء.
2. **Redis**: أنا أعيش في الذاكرة RAM، استخدم خوارزميات مثل LRU.

3. **Gossip Protocol**: أنا لا أملك قائداً؛ السر وراء قدرة Cassandra و Kubernetes على معرفة حالة العقد.
4. **Eventual Consistency**: أضحى بالدقة اللحظية (Strong Consistency) لأمنك التوفر والاستمرارية.

خوارزمية Raft (Raft Consensus Algorithm)

هي واحدة من أهم الخوارزميات المستخدمة في الأنظمة الموزعة (Distributed Systems) لضمان أن مجموعة من الأجهزة (Nodes) تتفق على حالة واحدة مشتركة، حتى في حالة تعطل بعضها أو حدوث مشاكل في الشبكة.

تم تصميمها لتكون أسهل في الفهم والتطبيق مقارنة بخوارزميات سابقة مثل Paxos، مع الحفاظ على نفس مستوى الأداء والموثوقية.

كيف تعمل خوارزمية Raft؟

تعتمد الخوارزمية على فكرة "القائد" (Leader) وتوزع العمل إلى ثلاث مهام رئيسية:

- 1. **انتخاب القائد (Leader Election)**
 - في أي لحظة، يجب أن يكون هناك قائد واحد فقط في النظام.
 - تستخدم Raft مؤقتات عشوائية (Randomized Timeouts). إذا لم يتلقَ أحد الأجهزة (التابعين - Followers) إشارة من القائد لفترة معينة، يبدأ هذا الجهاز "انتخابات" ليصبح هو القائد الجديد.
 - يطلب الجهاز التصويت من الأجهزة الأخرى، وإذا حصل على "أغلبية الأصوات" (Quorum)، يصبح هو القائد.

2. نسخ السجلات (Log Replication)

- القائد هو المسؤول الوحيد عن استقبال طلبات الكتابة من العميل (Client).
- يقوم القائد بإضافة هذه الطلبات إلى "سجله" (Log) الخاص، ثم يرسلها إلى باقي الأجهزة (التابعين).
- بمجرد أن تؤكد أغلبية الأجهزة أنها استلمت الطلب وكتبته في سجلاتها، يعتبر القائد هذا الطلب "مؤكدًا" (Committed) ويقوم بتنفيذه على الحالة العامة للنظام.

3. الأمان (Safety)

- تضمن Raft أن جميع الأجهزة ستنفذ نفس العمليات وبنفس الترتيب.
- لا يمكن لأي جهاز أن يصبح قائداً إلا إذا كان يحتوي على جميع السجلات المؤكدة من الفترات السابقة، مما يمنع فقدان البيانات.

الأنوار الأساسية (Node Roles)

أي جهاز في مجموعة Raft يكون في واحدة من ثلاث حالات:

- القائد (Leader)**: يستقبل الطلبات، يدير السجلات، ويرسل "نبضات قلب" (Heartbeats) للأجهزة الأخرى ليخبرهم بأنه لا يزال موجوداً.
- التابع (Follower)**: خامل ويستجيب فقط لطلبات القائد أو لطلبات التصويت.
- المرشح (Candidate)**: حالة مؤقتة يمر بها الجهاز عندما يبدأ عملية انتخاب ليصبح قائداً.

لماذا Raft مهمة؟

- التوافقية (Consistency)**: تضمن أن جميع النسخ في النظام الموزع ترى نفس البيانات.

- التسامح مع الأخطاء (Fault Tolerance): طالما أن الأغلبية (مثلاً 3 من أصل 5 أجهزة) تعمل، فإن النظام يستمر في العمل بشكل طبيعي.
- الوضوح: بفضل تقسيم الخوارزمية إلى أجزاء مستقلة (انتخاب، نسخ، أمان)، أصبح من الأسهل على المهندسين فهمها وتطبيقها في أنظمة حقيقية مثل etcd (المستخدم في Kubernetes) و Consul.

الأكواد

1. كود محاكاة زمن الاستجابة (Latency)

هذا الكود يمثل الخادم (Server) ويحاكي التأخير الناتج عن الشبكة أو العمليات الحسابية

```
public class StockServiceImpl extends UnicastRemoteObject implements IStockService {
    public double getPrice(String symbol) throws RemoteException {
        try {
            // Simulate 2 seconds of network/processing latency
            Thread.sleep(2000);
        } catch (InterruptedException e) { e.printStackTrace(); }

        return 150.00; // Return price after the delay
    }
}
```

الفكرة:

- يوضح الكود تأثير الكمون (Latency)؛ حيث يتم إجبار الخادم على الانتظار لمدة 2000 ميلي ثانية (2 ثانية) قبل إرجاع النتيجة.
- هذا التأخير يمثل العوائق الواقعية مثل المسافة الفيزيائية أو ازدحام الشبكة أثناء طلب البيانات.

2. كود معالجة الفشل الجزئي (Partial Failure)

يوضح هذا الكود كيفية التعامل برمجياً مع حالة "عدم اليقين" عند انقطاع الاتصال.

```

try {
    double price = stockService.getPrice("AAPL");
    System.out.println("Price is: " + price);
} catch (ConnectException e) {
    // Specific Partial Failure: The server is down or network is cut
    System.out.println("Server is unreachable. Switching to offline cache...");
} catch (RemoteException e) {
    // General Partial Failure: Something went wrong mid-call
    System.out.println("The call started but failed. Status uncertain.");
}

```

الفكرة:

- في الأنظمة الموزعة، قد يفشل جزء من النظام بينما يعمل الآخر.
- يستخدم **try-catch** للتعامل مع **الفشل الجزئي**؛ حيث يتم توفير بدائل (مثل استخدام الذاكرة المؤقتة) عند فشل الاتصال، أو إخطار النظام بأن حالة العملية غير مؤكدة (هل تم الدفع أم لا؟).

3. كود التزامن (Concurrency)

يوضح الكود كيفية منع حدوث "تضارب" عند محاولة حجز مورد وحيد (مثل تذكرة) من قبل مستخدمين في نفس الوقت.

```

public class TicketOffice {
    private int ticketsAvailable = 1;

    public void buyTicket(String userName) {
        if (ticketsAvailable > 0) {
            // Simulate a tiny delay in processing
            try { Thread.sleep(10); } catch (Exception e) {}

            ticketsAvailable--;
            System.out.println("Ticket sold to: " + userName);
        } else {
            System.out.println("Sold out for: " + userName);
        }
    }
}

```

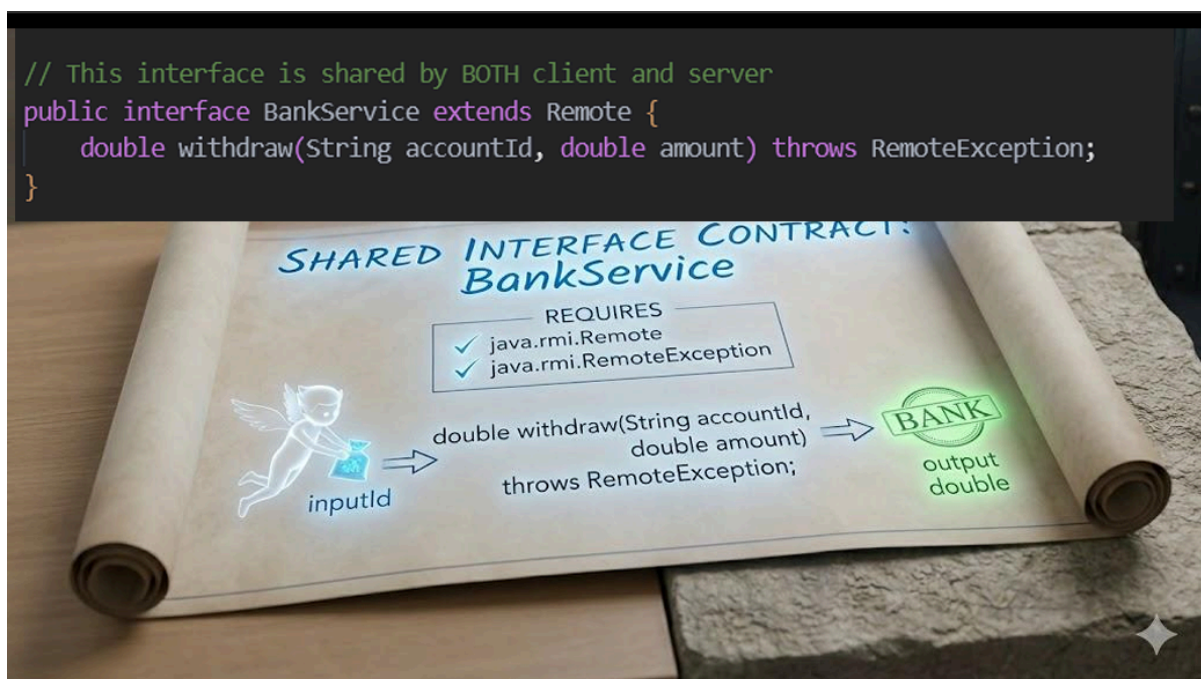
الفكرة:

- تكمن المشكلة في أن كود حجز التذكرة قد يعمل في خيوط معالجة (Threads) متزامنة، مما قد يؤدي لبيع نفس التذكرة لشخصين.

- استخدام الكلمة المفتاحية **synchronized** يضمن التزامن (Concurrency)؛ فهي تجعل الدالة محمية بحيث لا يمكن لأكثر من مستخدم تنفيذها في نفس اللحظة، مما يمنع حدوث "اصطدام" في الموارد المشتركة.

1. واجهة الخدمة (Service Interface)

هذا هو العقد (Contract) الذي يتفق عليه الخادم والعميل. يجب أن تراث الواجهة من **Remote**.



الفكرة:

- تحدد هذه الواجهة الخدمات المتاحة.
- استخدام **Remote** يخبر نظام RMI أن هذه الواجهة مخصصة للاستدعاء عن بُعد.
- إجبارية **RemoteException** هي وسيلة RMI لإجبار المبرمج على التعامل مع احتمالية فشل الشبكة أو تعطل الخادم أثناء الاستدعاء.

2. تنفيذ الخدمة (Server Implementation)

هذا الكود هو "العقل" الذي يعمل على الخادم ويقوم بتنفيذ المهمة الفعلية.

```

public class BankServer extends UnicastRemoteObject implements BankService {
    public double withdraw(String id, double amt) { /* ... logic ... */ }

    public static void main(String[] args) {
        // 1. Start a tiny lookup service on port 1099
        LocateRegistry.createRegistry(1099);

        // 2. Create the actual service object
        BankService service = new BankServer();

        // 3. "Register" it under a name so clients can find it
        Naming.rebind("rmi://localhost:1099/BankService", service);
    }
}

```

الفكرة:

- وراثته **UnicastRemoteObject** تجعل الكائن قابلاً للاستماع إلى الطلبات الواردة من العملاء عبر الشبكة.
- يقوم هذا الكائن بفتح "منفذ" (Port) وينتظر اتصالات من العملاء الذين يريدون تنفيذ الدالة **getPrice**.

3. جانب العميل (The Client)

كيف يستخدم العميل هذه الخدمة دون معرفة مكانها الفعلي.

```

public class BankClient {
    public static void main(String[] args) {
        // Look up the remote object by name (like a phonebook)
        BankService bank = (BankService) Naming.lookup("rmi://server-ip:1099/BankService");

        // Call it LIKE A LOCAL METHOD ✨
        double result = bank.withdraw("ACC123", 50.0);
        System.out.println("New balance: " + result);
    }
}

```

الفكرة:

- **Naming.lookup**: هي آلية البحث (مثل دفتر العناوين) التي تمكن العميل من العثور على الكائن البعيد باستخدام عنوانه واسمه (URL).
- بعد الحصول على المرجع (**stockService**)، لا يهتم العميل أين يوجد الخادم؛ هو فقط يستدعي الدالة، ويقوم نظام RMI خلف الكواليس بـ:
 1. تغليف البيانات (Marshaling).
 2. إرسالها عبر الشبكة.
 3. انتظار الرد وإعادة تجميعه (Unmarshaling).

أولاً: كود التسلسل (Serialization)

- الفكرة: تحويل كائن (Object) إلى بايتات (Bytes) ليتم إرساله عبر الشبكة، ثم إعادة تحويل البايتات إلى كائن في الطرف الآخر.
- الكود: يستخدم `Serializable` لتمكين الكائن من التسلسل، و `ObjectOutputStream` لتحويل الكائن إلى بايتات، و `ObjectInputStream` لاستعادته.

```
import java.io.*;

class Message implements Serializable {
    String text;
    Message(String t) { text = t; }
}

public class SerializationDemo {
    public static void main(String[] args) throws Exception {
        // 1 SERIALIZATION: Object → Byte Array
        Message original = new Message("Hello, World!");
        ByteArrayOutputStream bos = new ByteArrayOutputStream();
        ObjectOutputStream out = new ObjectOutputStream(bos);
        out.writeObject(original);
        byte[] bytes = bos.toByteArray();
        System.out.println("Serialized into " + bytes.length + " bytes");

        // 2 DESERIALIZATION: Byte Array → New Object
        ByteArrayInputStream bis = new ByteArrayInputStream(bytes);
        ObjectInputStream in = new ObjectInputStream(bis);
        Message copy = (Message) in.readObject();
        System.out.println("Deserialized text: " + copy.text);
    }
}
```

The RMI does this automatically

ثانياً: كود اكتشاف الخدمة (Service Discovery)

- الفكرة: ربط اسم منطقي (Logical Name) بخدمة معينة لتمكين العملاء من الوصول إليها دون الحاجة لمعرفة عنوان الـ IP والمنفذ (Port) بشكل مباشر.
- الكود: يستخدم `Registry` للقيام بعملية `rebind` (الخادم يسجل الخدمة) و `lookup` (العميل يبحث عن الخدمة).

Last time

```
// SERVER: Put object under a name
Registry reg = LocateRegistry.createRegistry(1099);
reg.rebind("PaymentService", new PaymentImpl());

// CLIENT: Ask registry for the name
Registry reg = LocateRegistry.getRegistry("server-ip", 1099);
PaymentService svc = (PaymentService) reg.lookup("PaymentService");
```

The RMI Registry is your discovery service. Clients bind to a logical name, not a hard IP:Port.

ثالثاً: تمرير القيم والمراجع (Pass-by-value vs Pass-by-reference)

- الفكرة: في RMI:
 - تمرير بالقيمة (Serializable/POJO): يتم إرسال نسخة طبق الأصل (Photocopy) من الكائن. أي تعديل على النسخة في العميل لا يؤثر على الخادم.
 - تمرير بالمراجع (Remote): يتم إرسال "Stub" (رابط)، وكل استدعاء يتم على هذا الكائن ينتقل فعلياً للخادم وينفذ هناك.

Last time

```
interface RemoteAPI extends Remote {
    User getUser() throws RemoteException;           // Pass-by-Value
    RemotePrinter getPrinter() throws RemoteException; // Pass-by-Reference (Stub)
}

// CLIENT USAGE
User u = api.getUser();           // LOCAL COPY. Changing it here does NOTHING to the server.
u.name = "Bob";

RemotePrinter p = api.getPrinter(); // REMOTE STUB. This call travels to the server.
p.print("Hello");                  // Actually executes on the remote machine.
```

Normal objects cross the network as copies (value). Objects that implement **Remote** cross as stubs that act like references back to the server.

رابعاً: موازنة الأحمال (Load Balancing)

تتمثل الفكرة الأساسية في توزيع الطلبات الواردة على مجموعة من الخوادم لتجنب الضغط الزائد على خادم واحد، وتضمنت المحاضرة ثلاثة حلول رئيسية:

- **الحل الأول: أقل الاتصالات نشطاً (Fewest Active Connections)**
 - **الفكرة:** يتم توجيه كل طلب جديد إلى الخادم الذي يمتلك حالياً أقل عدد من الاتصالات النشطة.
 - **الهدف:** التوزيع الذكي بناءً على الحمل الفعلي (Load) وليس التوزيع العشوائي.
- **الحل الثاني: خوارزمية الاختيار العشوائي الذكي (Power of Two Choices)**
 - **الفكرة:** يتم اختيار خادمين عشوائياً من القائمة، ثم المقارنة بينهما واختيار الأقل ضغطاً بينهما لتنفيذ المهمة.
 - **الهدف:** تحقيق توازن شبه مثالي بتكلفة حسابية منخفضة جداً ($O(1)$))، وتجنب ظهور "النقاط الساخنة" (Hot spots).
- **الحل الثالث: المراقبة المستمرة (Continuous Monitoring)**
 - **الفكرة:** لا تُعتبر خوارزمية توجيه فحسب، بل هي "نمط بنيوي" (Infrastructure Pattern). تعتمد على دمج مراقبة صحة الخوادم (Health Checks) مع قوائم الخوادم المتاحة ديناميكياً.
 - **الهدف:** لا يتم توجيه الطلبات إلا للخوادم التي أثبتت أنها "حية" وقادرة على الاستجابة، مما يجعل النظام أكثر مرونة وتحملاً للأعطال.

```
# Tracks real-time load: {server_name: active_connections}
servers = {"A": 5, "B": 12, "C": 2}

def route_least_connections():
    # Greedy: pick the minimum right now
    chosen = min(servers, key=servers.get)

    servers[chosen] += 1 # Connection starts
    return chosen

# Later, when request finishes:
# servers[chosen] -= 1
```

What it fixes: Routes each new request to the server with the fewest active connections.

What it fixes: Scanning *all* servers is slow under heavy traffic. Picking 2 at random and comparing them gives near-optimal balance with $O(1)$ cost.

```
import random

servers = {"A": 5, "B": 12, "C": 2}

def route_power_of_two():
    # 1. Randomly pick 2 different servers
    candidates = random.sample(list(servers.keys()), 2)
    # 2. Greedy: pick the lighter of the two
    chosen = min(candidates, key=lambda s: servers[s])

    servers[chosen] += 1
    return chosen
```

Why it's better: Proven mathematically to prevent hot spots with minimal overhead. Used by Twitter, Uber, and AWS internally.

This is **not** a routing algorithm. It's an infrastructure pattern. You cannot fix it with a single routing function; you need continuous monitoring + dynamic server lists.

Here's how a load balancer *consumes* health data before routing:

```
# Health tracker (updated by heartbeat agents every 5s)
server_health = {"A": True, "B": False, "C": True}
server_load    = {"A": 5, "B": 12, "C": 2}

def route_with_health():
    # 1. Filter out dead/sick servers
    healthy = [s for s, alive in server_health.items() if alive]
    if not healthy: return "FALLBACK/QUEUE"

    # 2. Apply greedy routing ONLY to healthy nodes
    chosen = min(healthy, key=lambda s: server_load[s])
    server_load[chosen] += 1
    return chosen
```

خامساً: التمرير بالمرجع (Pass-by-reference / Remote Object)

في برمجة Java العادية، نحن نمرر الكائنات عادةً بالقيمة (Pass-by-value). ولكن في RMI، تختلف الأمور عند التعامل مع "الكائنات البعيدة" (Remote Objects):

- **الفكرة:** بدلاً من إرسال نسخة كاملة من الكائن (مثلما نفعل في التمرير بالقيمة)، يتم إرسال "رابط" أو "Stub" فقط. هذا الرابط يعمل كوكيل (Proxy) يشير إلى الكائن الأصلي الموجود على الخادم.
- **كيف يعمل:**
 - عندما يستدعي العميل أي "دالة" (Method) على هذا الكائن، لا يتم تنفيذ الكود محلياً عند العميل.
 - يتم إرسال طلب عبر الشبكة إلى الخادم، ويقوم الخادم بتنفيذ الكود على الكائن الأصلي الموجود في ذاكرته، ثم يعيد النتيجة للعميل.
- **متى نستخدمه؟** نستخدم التمرير بالمرجع عندما نريد أن يكون هناك نسخة واحدة فقط من الكائن على الخادم ليتشارك الجميع في تعديل حالته، أو إذا كان الكائن كبيراً جداً ولا نريد تحميل الشبكة بإرساله كاملاً.

ملخص سريع للفرق :

وجه المقارنة	التمرير بالقيمة (Pass-by-value)	التمرير بالمرجع (Pass-by-reference)
ما الذي يُرسل؟	نسخة كاملة من بيانات الكائن.	رابط (Stub) فقط.
أين يُنفذ الكود؟	عند العميل (على نسخته الخاصة).	عند الخادم (على النسخة الأصلية).
التأثير	التغييرات لا تنعكس على الخادم.	التغييرات تنعكس على الكائن الأصلي.

عند العميل: الـ **Stub** يغلف الطلب (Marshalling).
عبر الشبكة: انتقال الطلب إلى الخادم.
عند الخادم: الـ **Skeleton** يفك التغليف وينفذ العملية على الكائن الحقيقي.

```
public class GreedyRMIRegistry {
    private final Map<String, ServerNode> servers = new ConcurrentHashMap<>();

    public static class ServerNode {
        String rmiUrl;
        AtomicInteger activeCalls = new AtomicInteger(0);
        public ServerNode(String rmiUrl) { this.rmiUrl = rmiUrl; }
    }

    public void registerServer(String url) {
        servers.put(url, new ServerNode(url));
    }

    public <T extends Remote> T getLeastLoaded(Class<T> iface) throws Exception {
        ServerNode chosen = servers.values().stream()
            .min(Comparator.comparingInt(n -> n.activeCalls.get()))
            .orElseThrow();

        chosen.activeCalls.incrementAndGet();
        Registry reg = LocateRegistry.getRegistry(new URI(chosen.rmiUrl).getHost(), 1099);
        T proxy = iface.cast(reg.lookup(iface.getSimpleName()));

        // Wrap proxy to decrement on completion (use dynamic proxy or AOP in real code)
        return proxy;
    }
}
```

شرح خوارزمية Weighted Round-Robin

تُستخدم هذه الخوارزمية في موازنة الأحمال (Load Balancing) لتوزيع الطلبات القادمة على مجموعة من الخوادم (Servers)، ليس بالتساوي، بل بناءً على "وزن" أو قدرة كل خادم. الخادم الذي يمتلك وزناً أكبر سيستقبل عدداً أكبر من الطلبات مقارنة بالخادم الذي يمتلك وزناً أقل.

1. حساب الوزن الكلي (**totalWeight**): يقوم الكود بجمع كافة الأوزان المحددة لكل الخوادم المتاحة.
2. تحديد المؤشر الحالي (**current**): يتم استخدام عداد (**index**) يزداد مع كل طلب، ثم نأخذ باقي القسمة على الوزن الكلي (**totalWeight %**). هذا يضمن أننا نتحرك دائماً في نطاق دوري ضمن مجموع الأوزان.
3. التوزيع التراكمي:
 - يبدأ الكود بالمرور على الخوادم واحداً تلو الآخر.
 - يقوم بجمع أوزان الخوادم بشكل تراكمي في متغير (**weightSum**).
 - بمجرد أن يصبح الرقم الحالي (**current**) أقل من المجموع التراكمي للوزن (**weightSum**)، يتم اختيار هذا الخادم فوراً.
4. النتيجة: هذا يضمن أن الخادم الذي له وزن أكبر سيشغل "مساحة" أوسع في نطاق الأرقام التراكمية، مما يزيد من احتمالية اختياره.

مثال توضيحي:

- إذا كان الخادم A وزنه 2، والخادم B وزنه 1.
- المجموع الكلي = 3.
- الطلبات ستوزع بنمط (A, A, B) بشكل متكرر، وهكذا يحصل A على حصة أكبر من الطلبات.

```
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class WeightedRoundRobin {
    static class Server { String name; int weight; Server(String n, int w) { name=n; weight=w; } }
    private final List<Server> servers;
    private final AtomicInteger index = new AtomicInteger(0);

    public WeightedRoundRobin(List<Server> servers) { this.servers = servers; }

    public Server select() {
        int totalWeight = servers.stream().mapToInt(s -> s.weight).sum();
        int current = index.getAndIncrement() % totalWeight;
        int weightSum = 0;
        for (Server s : servers) {
            weightSum += s.weight;
            if (current < weightSum) return s;
        }
        return servers.get(0);
    }
}
```

خوارزمية Power of Two Choices

بدلاً من اختيار خادم واحد عشوائياً (الذي قد يكون مشغولاً جداً) أو البحث عن "الأقل انشغالاً" من بين جميع الخوادم (الذي يستهلك طاقة معالجة عالية)، تقوم هذه الخوارزمية بـ:

1. اختيار **خادمين اثنين** بشكل عشوائي.
2. مقارنة "حمل" هذين الخادمين.
3. إرسال الطلب إلى الخادم الذي يمتلك **الحمل الأقل** بينهما.
4. **الاختيار العشوائي (s1 و s2)**: يقوم الكود باستخدام `ThreadLocalRandom` لاختيار خادمين (`s1` و `s2`) من قائمة الخوادم المتاحة بشكل عشوائي تماماً.
5. **المقارنة (s1.load < s2.load)**: يقارن الكود قيمة الحمل (أو عدد الطلبات الحالية) الموجودة في كل من الخادمين المختارين.
6. **اتخاذ القرار**: يقوم بإرجاع الخادم الذي لديه قيمة حمل أقل (`s1` إذا كان أقل، وإلا `s2`).

لماذا تُستخدم هذه الخوارزمية؟

تعتبر هذه الخوارزمية ممتازة لأنها:

- **تتجنب "التكدس"**: من خلال اختيار الأقل بين اثنين، تضمن توزيعاً متوازناً جداً دون الحاجة لمراقبة جميع الخوادم في كل مرة.
- **الأداء العالي**: تقليل البحث إلى خادمين فقط يوفر وقتاً كبيراً مقارنة بالبحث عن الخادم الأقل انشغالاً من بين مئات الخوادم.

- **بسيطة التنفيذ:** لا تتطلب بنية تحتية معقدة لجمع بيانات كافة الخوادم في وقت واحد.

Coding Example

```
import java.util.SortedMap;
import java.util.TreeMap;

public class ConsistentHashing {
    private final SortedMap<Integer, String> ring = new TreeMap<>();

    public void addServer(String server) { ring.put(server.hashCode(), server); }
    public void removeServer(String server) { ring.remove(server.hashCode()); }

    public String getServer(String requestKey) {
        Integer key = requestKey.hashCode();
        SortedMap<Integer, String> tailMap = ring.tailMap(key);
        return tailMap.isEmpty() ? ring.get(ring.firstKey()) : ring.get(tailMap.firstKey());
    }
}
```

خوارزمية Least Connections

في الأنظمة التي تتفاوت فيها مدة معالجة الطلبات (بعض الطلبات سريعة وبعضها يستغرق وقتاً طويلاً)، لا يكفي توزيع الطلبات بالتساوي (مثل Round-Robin)، بل يجب إرسال الطلب الجديد للخادم الذي لديه **أقل عدد من الطلبات النشطة حالياً**.

1. الافتراض الأولي: يبدأ الكود بفرض أن الخادم الأول في القائمة (`servers.get(0)`) هو الأقل انشغالاً (`leastLoaded`).
2. البحث (`Iteration`): يقوم الكود بعمل حلقة تكرارية (`for`) للمرور على جميع الخوادم المتاحة.
3. المقارنة: في كل خطوة، يقارن عدد الاتصالات النشطة (`activeConnections`) للخادم الحالي بـ `leastLoaded`. إذا كان الخادم الحالي يمتلك عدداً أقل من الاتصالات، يتم تحديث المتغير `leastLoaded` ليكون هو هذا الخادم.
4. النتيجة: بعد انتهاء الحلقة، يعيد الكود الخادم الذي تبين أنه يمتلك أقل عدد من الاتصالات النشطة، ليقوم باستقبال الطلب الجديد.

لماذا تُستخدم هذه الخوارزمية؟

- **تجنب التحميل الزائد:** تضمن أن الخوادم القوية أو الخوادم التي أنهت مهامها بسرعة تستقبل طلبات جديدة، بينما يتم تجنب الخوادم المزدحمة التي لا تزال تعالج طلبات طويلة الأمد.
- **مناسبة للتطبيقات الحية:** فعالة جداً في تطبيقات الويب أو الـ Streaming حيث تختلف أوقات استجابة الخوادم بناءً على نوع الطلب.

```
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

public class StickySessionLB {
    private final List<String> servers;
    private final Map<String, String> sessionMap = new ConcurrentHashMap<>();
    private final AtomicInteger rrIndex = new AtomicInteger(0);

    public StickySessionLB(List<String> servers) { this.servers = servers; }

    public String select(String sessionId) {
        return sessionMap.computeIfAbsent(sessionId, k ->
            servers.get(rrIndex.getAndIncrement() % servers.size()));
    }
}
```

شرح خوارزمية Least Response Time

يقوم الكود باختيار الخادم (Server) الذي يمتلك أقل متوسط زمن استجابة لاستقبال الطلبات القادمة.

1. هيكلية البيانات (Server class)

يحتوي كل خادم على ثلاثة بيانات أساسية:

- **id**: معرف الخادم.
- **totalLatency**: مجموع أزمنة الاستجابة التي سجلها هذا الخادم حتى الآن.
- **count**: عدد الطلبات التي تم التعامل معها بواسطة هذا الخادم.

2. تخزين الخوادم

يستخدم الكود **ConcurrentHashMap** لتخزين الخوادم، وهو خيار ممتاز في بيئات البرمجة متعددة الخيوط (Multi-threaded) لضمان سلامة البيانات ومنع حدوث تضارب عند القراءة والكتابة في نفس الوقت.

3. الدوال الرئيسية:

- **add(String id)**: تُستخدم لإضافة خادم جديد إلى القائمة.
- **recordLatency(String serverId, long ms)**: تُستدعى عند انتهاء معالجة طلب ما، حيث تقوم بتحديث إجمالي زمن الاستجابة (**totalLatency**) وزيادة عدد الطلبات (**count**) للخادم المعني.
- **select()**: هي قلب الموازن. تقوم بحساب متوسط زمن الاستجابة لكل خادم $\frac{\text{totalLatency}}{\text{count}}$ ، ثم تستخدم **Stream API** للبحث عن الخادم الذي لديه أقل قيمة (Minimum) ليعيدها كخادم مثالي لاستقبال الطلب التالي.

Coding Example

```
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class LatencyBasedLB {
    static class Server { String id; long totalLatency; int count; }
    private final Map<String, Server> servers = new ConcurrentHashMap<>();

    public void add(String id) { servers.put(id, new Server()); servers.get(id).id = id; }

    public String select() {
        return servers.values().stream()
            .min((a, b) -> Double.compare((double)a.totalLatency/a.count, (double)b.totalLatency/b.count))
            .map(s -> s.id).orElse(null);
    }

    public void recordLatency(String serverId, long ms) {
        Server s = servers.get(serverId);
        s.totalLatency += ms; s.count++;
    }
}
```

خوارزمية "الخيار المعتمد على الحالة" (Stateful/Sticky Session - Hash Load Balancing)، وهي تُستخدم عندما نحتاج لضمان توجيه طلبات مستخدم معين (Session) إلى نفس الخادم في كل مرة.

شرح خوارزمية (Hash (Sticky Session

في بعض التطبيقات (مثل سلة التسوق في المواقع)، يجب أن يعرف الخادم حالة المستخدم السابقة. إذا تم تغيير الخادم مع كل طلب، ستفقد معلومات الجلسة (Session). هنا نستخدم "Hash" لربط معرف المستخدم (Client IP أو Session ID) بخادم معين بشكل دائم.

Coding Example #2

```
latencies = {"A": 50, "B": 120, "C": 30} # ms
ALPHA = 0.2 # Smoothing factor

def update_latency(server, measured_ms):
    latencies[server] = ALPHA * measured_ms + (1 - ALPHA) * latencies[server]

def route_latency_based():
    return min(latencies, key=latencies.get)
```


كود Nginx Load Balancer

هذا الكود هو إعداد أساسي (Configuration) لجعل Nginx يعمل كموزع للأحمال (Load Balancer) بين خادمين للتطبيقات:

1. `upstream my_app { ... }`
 - هذا الجزء يحدد مجموعة من الخوادم الخلفية (Backend Servers) التي سيتم توزيع الطلبات عليها.
 - تم تعريف خادمين هنا يعملان على العناوين `127.0.0.1:8081` و `127.0.0.1:8082`.
2. `server { listen 80; ... }`
 - يخبر Nginx بالاستماع للطلبات الواردة على المنفذ الافتراضي 80 (الخاص بـ HTTP).
3. `location / { ... }`
 - يحدد أن جميع الطلبات القادمة إلى المسار الرئيسي (/) يجب معالجتها بالقواعد التالية:
 - `proxy_pass http://my_app;` يقوم بتوجيه (تمرير) الطلبات إلى مجموعة الخوادم التي سمينها `my_app` أعلاه، حيث سيقوم Nginx بتوزيعها بين الخادمين المحددين.
4. `proxy_set_header ...`
 - تقوم هذه الأسطر بتمرير معلومات العميل الأصلية (مثل عنوان IP الخاص بالمستخدم واسم النطاق المطلوب) إلى الخوادم الخلفية، حتى تعرف هذه الخوادم من الذي يطلب الخدمة بالضبط بدلاً من أن تظن أن الطلب قادم من Nginx فقط.

```
http {
    # تعريف مجموعة الخوادم (Upstream)
    upstream my_app {
        server 127.0.0.1:8081;
        server 127.0.0.1:8082;
    }

    server {
        listen 80;

        location / {
            # توجيه الطلبات للمجموعة المعرفة أعلاه
            proxy_pass http://my_app;

            # تمرير معلومات العميل للخوادم الخلفية
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```